ECON662_HW2

April 6, 2026

[2]:
```
import time
import numpy as np
import pandas as pd
from scipy.optimize import minimize
from scipy.special import logsumexp, ndtr, expit
from scipy.linalg import solve, lu_factor, lu_solve
from scipy.stats import norm
import random
import sys
import subprocess
from pathlib import Path
from sklearn.linear_model import LogisticRegression

# if you are using the google colab
# from google.colab import drive
# drive.mount('/content/drive')

random.seed(42)
```

1 Homework 2 ECON 662

Group Member:

Xuyuan Zhang

Paola Villa Paro

Wulan Kencana

We maintain the information on the github:

https://github.com/pvillaparo/ECON662-HW2

We

1.1 State Transition Matrix and State Space

Here I want to consider the detailed information on the dynamic question

The state space is given by $s = (x, rc)$ where $x \in \{1, \dots, 7\}$ and rc is the replacement cost. Action space $a = \{0, 1\}$ denoting replacing or not replacing. In solving, we denote G RC points, then the

total number of states is given by $S = 7G$.

$$RC_t = \rho_0 + \rho_1 RC_{t-1} + e_t$$

The transition matrix is given by:

$$Q = [Q_{gh}]_{g,h=1}^{G}, \quad Q_{gh} = \Pr(RC_{t+1} \in \text{ grid } h \mid RC_t = r^g)$$

Denote grid space as Δ then the interior grid cells is:

$$Q_{gh} = \Phi\left(\frac{r^h + \Delta/2 - (\rho_0 + \rho_1 r^g)}{\sigma_\rho}\right) - \Phi\left(\frac{r^h - \Delta/2 - (\rho_0 + \rho_1 r^g)}{\sigma_\rho}\right)$$

with the corresponding tail formulas for the first and last bins.

In terms of Mileage transition, we can consider the transition is given by:

$$x' = \begin{cases} 1 & \text{if } a = 1 \\ \min(7, x+1) & \text{if } a = 0 \end{cases}$$

We denote $M_0(x, x') = \mathbf{1}\{x' = x' = \min(7, x+1)\}$ and $M_1(x, x') = \mathbf{1}\{x' = 1\}$. Then the full state transition is given by:

$$F_0 = M_0 \otimes Q, \quad F_1 = M_1 \otimes Q$$

Therefore, for current $s = (x, rc)$ and the next state, we have:

$$F_a(s, s') = \Pr(s' \mid s, a)$$

1.1.1 Bellman Equation

Let $V(s)$ be the ex ante function. We can write the action specific continuation value as:

$$EV_a(s) = \sum_{s'=1}^{S} F_a(s, s') V(s')$$

Then the Bellman Equation is given by:

$$V(s) = \mathbb{E}_\varepsilon[\max_{a \in \{0,1\}} (\bar{u}(s, a) + \varepsilon_a + \beta EV_a(s))]$$

where the stock is a Type-1 extreme value. Then we can write as:

$$V(s) = \log(\exp(\bar{u}(s, 0) + \beta EV_0(s)) + \exp(\bar{u}(s, 1) + \beta EV_1(s))) + \gamma$$

(Side Note: Here I leverage the term from the fact of Appendix A)

where γ is a Euler Constant. Then we can substitute the utility term and we have:

$$V(s) = \log(\exp\{-\theta_1 x_s + \beta EV_0(s)\} + \exp\{-\theta_2 rc_s + \beta EV_1(s)\}) + \gamma$$

(Of course, if we don't normalize the σ_a the result should be):

$$V(s) = \sigma_a \gamma + \sigma_a \log(\exp(\frac{\bar{u}(s,0) + \beta EV_0(s)}{\sigma_a}) + \exp(\frac{\bar{u}(s,1) + \beta EV_1(s))}{\sigma_a}))$$

1.2 CCP and NXFP

In the logit CCP implied by Bellman Equation, we can write

$$\Pr(a = 1 \mid s) = \frac{\exp(-\theta rc_s + \beta EV_1(s))}{\exp\{-\theta_1 x_s + \beta EV_0(s)\} + \exp\{-\theta_2 rc_s + \beta EV_1(s)\}}$$

and

$$\Pr(a = 0 \mid s) = \frac{\exp\{-\theta_1 x_s + \beta EV_0(s)\}}{\exp\{-\theta_1 x_s + \beta EV_0(s)\} + \exp\{-\theta_2 rc_s + \beta EV_1(s)\}}$$

Then we can write the State-level likelihood, which aggregates the likelihood to the state level. Once the state s is fixed, we can write

$$N_1(s) = \#\{i : a_i = 1 \text{ in state } s\}, \quad N_0(s) = \#\{i : a_i = 0 \text{ in state } s\}$$

Then the log-likelihood can be written as:

$$\ell(\theta) = \sum_{s=1}^{S} [N_1(s) \log \Pr(a = 1 \mid s; \theta) + N_0(s) \log \Pr(a = 0 \mid s; \theta)]$$

We can write the nested fixed point algorithm using the below condition:

We can denote

$$EV = \begin{pmatrix} EV_0 \\ EV_1 \end{pmatrix}$$

and define the Bellman operator T_θ. Then the fixed point is given by:

$$EV = T_\theta(EV)$$

and the numerical expression ig: $EV^k = T_\theta(EV^{k-1})$. By the Banach's fixed point theorem, we can have: $EV^k \to EV_\theta$.

1.3 Hotz-Miller Inversion

Based on the Bellman Equation and we can write it using the CCP condition as:

$$V(s) = \sum_{a\in\{0,1\}} P(a \mid s)[\bar{u}(a,s) + \beta \sum_{s'} F_a(s,s')V(s')] + \sum_{a\in\{0,1\}} P(a \mid s)\mathbb{E}[\varepsilon_a \mid a \text{ chosen } s].$$

Under the Type-1 extreme value assumption we have:

$$\mathbb{E}[\varepsilon_a \mid a \text{ chosen }, s] = \gamma - \log P(a \mid s)$$

Therefore,

$$V(s) = \sum_{a\in\{0,1\}} P(a \mid s) \left[\bar{u}(s,a) + \beta \sum_{s'} F_a(s,s')V(s') + \gamma - \log P(a \mid s) \right]$$

Now write in the vector form and we can write

$$P_a = \begin{pmatrix} P(a \mid 1) \\ \cdots \\ P(a \mid S) \end{pmatrix} \quad \bar{u}_a = \begin{pmatrix} \bar{u}(1,a) \\ \cdots \\ \bar{u}(S,a) \end{pmatrix}$$

We can write the diagonal matrix whose diagonal entries are given P_a. Then the Bellman equation becomes

$$V = \sum_{a\in\{0,1\}} D(P_a)(\bar{u}_a + \gamma\mathbb{1} - \log P_a) + \beta \sum_a D(P_a)F_a V.$$

Move to the LHS, inverse it and we have:

$$V = \left[I - \beta \sum_a D(P_a)F_a \right]^{-1} \sum_a D(P_a)(\bar{u}_a + \gamma\mathbb{1} - \log P_a)$$

1.4 Part (a) and (b) Implementations

State space is constructed with $K = 8$ equal-width bins over the observed support for RC_t. The first optimization was to collapse the raw panel into state-level sufficient statistics. Instead of evaluating the log-likelihood observation by observation, the code first maps each observation into a state index and then counts how many replacement and non-replacement decisions occur in each state.

In terms of the optimization of the code, I start with the value function which is initialized from the solution found at the previous parameter vector, rather than restarting from zero each time. The main advantage is that adjacent parameter values usually imply adjacent value functions, so the next Bellman solve often starts very close to the new fixed point. This reduces the number of inner iterations needed during the outer maximization.

The third optimization was to replace matrix-free transition application by precomputed dense transition matrices F_0 and F_1. The full transition structure is built once in matrix form, and continuation values are then computed by simple matrix multiplication:

$$EV_0 = F_0 V, \quad EV_1 = F_1 V$$

Instead of reconstructing transition operations every time the Bellman operator is evaluated, the code performs one BLAS-style matrix multiplication, which is much faster in NumPy

I also use a flattened vector form in terms of the Bellman equation calculations. The main advantage is that the Bellman map, the continuation values, and the derivative objects are all expressed in a single consistent state-space representation. This reduces reshaping overhead and makes the code easier for NumPy and SciPy to handle efficiently.

I don't use the naive outer optimizer like the L-BFGS-B since the outer iteration only needs the curvature information from function evaluations. I consider a implicit differentiation of the Bellman fixed point:

$$(I - \frac{\partial T}{\partial V})\frac{\partial V}{\partial \theta_j} = \frac{\partial T}{\partial \theta_j}$$

Instead of repeatedly probing the objective numerically, the optimizer is given the local slope directly. This typically reduces the number of outer iterations and can materially speed up estimation when each likelihood evaluation is expensive.

I also revise the optimization to use a two-stage tolerance rule in NFXP. The first optimization stage solves the Bellman equation with a looser tolerance, and the second stage refines the estimate with a tighter tolerance. The main advantage is that the code does not waste time solving the Bellman equation very accurately at parameter values that are still far from the optimum. Early iterations are made cheap; only the final stage is made precise.

In terms of the Hotz-Miller optimization, I use the LU factorization to solve the linear system, which is very common in optimization theory. Because the CCPs are fixed in the HM step, the coefficient matrix is fixed as well, so it can be factorized once and reused.

Lastly, I also consider the smooth method in the code. Because I tried to increase $K = 8$ to $K = 50$ and I found that the estimation would fail because of the some of the states, there would be no samples drop in that place. Therefore, I consider a smooth method in the implementation of the code, and I found that the result is robust in terms of the K number.

In Paola's version, there is also an associated Tauchen's method in the estimation.

Tauchen is needed when your grid changes with parameters — i.e., when the AR(1) parameters are inside the optimizer. Here, Π is built from the **data directly** and is never recomputed. The grid is fixed, so no Tauchen formula is needed.

	Empirical (this notebook)	Tauchen
Grid built from	Data (bins)	AR(1) theory
Π recomputed?	Never	Every optimizer call
Params in optimizer	2	5

	Empirical (this notebook)	Tauchen
Empty-cell problem?	Yes (rare states)	No
Speed	Fast	Slower

[3]:
```
# basic setup

# if you are using the google colab
# DATA_PATH = '/content/drive/MyDrive/ECON662HWs/ddc_pset.csv'
DATA_PATH = 'ddc_pset.csv'

beta = 0.95
K = 8
ccp_clip = 1e-10                    # numerical clip so probabilities never hit␣
↪exactly 0 or 1

# First-stage CCP smoothing controls
# alpha is a small pseudo-count; Sx and Sr provide local averaging.
alpha_ccp = 1.0
w_center = 0.50
w_neighbor = 0.25

bellman_tol = 1e-10
bellman_tol_stage1 = 1e-6
bellman_tol_stage2 = 1e-11
bellman_maxiter = 5000
gamma_euler = 0.5772156649015329

theta0 = np.array([0.30, 0.35], dtype=np.float64)   # initial guess for␣
↪(theta1, theta2)

# Load data and organize panel

df = pd.read_csv(DATA_PATH).sort_values(["i", "t"]).reset_index(drop=True)
print(df.columns)

x_panel = df.pivot(index="i", columns="t", values="x").to_numpy(dtype=np.int16)
a_panel = df.pivot(index="i", columns="t", values="a").to_numpy(dtype=np.int8)

rc_path = (
    df[["t", "rc"]]
    .drop_duplicates(subset="t")
    .sort_values("t")["rc"]
    .to_numpy(dtype=np.float64)
)

N, T = x_panel.shape
```

```python
x_flat = x_panel.ravel(order="C").astype(np.int16) - 1
a_flat = a_panel.ravel(order="C").astype(np.int8)

# First stage for RC_t = rho0 + rho1 * RC_{t-1} + e_t

Y = rc_path[1:]
X = np.column_stack([np.ones(T - 1), rc_path[:-1]])
# Run OLS using numpy least squares.
rho_ols = np.linalg.lstsq(X, Y, rcond=None)[0]

# Pull out the intercept estimate.
rho0_hat = float(rho_ols[0])
# Pull out the persistence estimate and clip it slightly away from |rho1| >= 1
# to avoid numerical issues in subsequent transition calculations.
rho1_hat = float(np.clip(rho_ols[1], -0.999, 0.999))


# Compute OLS residuals e_t.
resid = Y - X @ rho_ols
# Estimate sigma_rho as the residual standard deviation.
sigma_rho_hat = float(np.sqrt(np.mean(resid ** 2)))
sigma_rho_hat = max(sigma_rho_hat, 1e-12)

print("First-stage estimates for RC_t:")
print(f"rho0_hat      = {rho0_hat:.6f}")
print(f"rho1_hat      = {rho1_hat:.6f}")
print(f"sigma_rho_hat = {sigma_rho_hat:.6f}")

# Discretize RC and aggregate state counts

rc_min = float(rc_path.min())
rc_max = float(rc_path.max())

# Build K+1 equally spaced bin edges covering the observed RC support.
edges = np.linspace(rc_min, rc_max, K + 1)
# Use the midpoint of each bin as the representative RC grid value.
r_grid = 0.5 * (edges[:-1] + edges[1:])


# For each month t, assign observed RC_t to one of the K bins.
# searchsorted(..., side="right") - 1 implements interval assignment.
rc_bin_by_month = np.searchsorted(edges, rc_path, side="right") - 1
# Clip to make sure the assigned bin index stays in {0,...,K-1}.
rc_bin_by_month = np.clip(rc_bin_by_month, 0, K - 1).astype(np.int16)

# Repeat the month-level RC bin sequence once for each bus.
# Because x_flat and a_flat were created in bus-major row order,
```

```python
# this tiled vector lines up exactly with those flattened panels.
rc_bin_flat = np.tile(rc_bin_by_month, N)

S = 7 * K
# Encode each state (x, g) as a single integer state id.
# Since x_flat is 0,...,6 and g is 0,...,K-1, the formula x*K + g is one-to-one.
state_id = x_flat * K + rc_bin_flat

# Count how many times action a=1 is observed in each state.
N1_vec = np.bincount(state_id[a_flat == 1], minlength=S).astype(np.float64)
# Count how many times action a=0 is observed in each state.
N0_vec = np.bincount(state_id[a_flat == 0], minlength=S).astype(np.float64)
N_vec = N1_vec + N0_vec
# Boolean mask for states that are actually observed in the data.
occupied = N_vec > 0

N1_mat = N1_vec.reshape(7, K)
N0_mat = N0_vec.reshape(7, K)
N_mat = N_vec.reshape(7, K)

# Empirical CCP = observed replacement frequency in each state.
empirical_ccp = np.divide(
    N1_vec,
    N_vec,
    out=np.zeros_like(N1_vec),
    where=(N_vec > 0)
)

# Print how many of the S states are actually visited.
print(f"\nOccupied states = {int(occupied.sum())} out of {S}")


# I tried this to use the $K = 50, 100$ and I find that if I don't smooth this
# the result will bias the whole estimation because there are a lot of states
# to be empty.

# Build a one-dimensional local averaging matrix of size n x n.
# Each state puts weight w_center on itself and w_neighbor on each immediate␣
↪neighbor.
# Rows are normalized to sum to 1.
def local_smoother_matrix(n, w_center=0.50, w_neighbor=0.25):
    M = np.zeros((n, n), dtype=np.float64)
    idx = np.arange(n)
    M[idx, idx] = w_center # own-state weight
    M[idx[:-1], idx[1:]] = w_neighbor # right neighbor weight
    M[idx[1:], idx[:-1]] = w_neighbor # left neighbor weight
    M /= M.sum(axis=1, keepdims=True) # row normalization
```

```
    return M

# Smoother along the mileage dimension.
Sx = local_smoother_matrix(7, w_center=w_center, w_neighbor=w_neighbor)
Sr = local_smoother_matrix(K, w_center=w_center, w_neighbor=w_neighbor)

# Smooth replacement counts over nearby mileage and nearby RC states.
N1_smooth = Sx @ N1_mat @ Sr.T
N_smooth = Sx @ N_mat @ Sr.T

# Convert smoothed counts into probabilities using pseudo-counts.
ccp_smooth_mat = (N1_smooth + alpha_ccp) / (N_smooth + 2.0 * alpha_ccp)

# Clip probabilities away from 0 and 1 for numerical stability.
ccp_smooth_mat = np.clip(ccp_smooth_mat, ccp_clip, 1.0 - ccp_clip)
ccp_smooth = ccp_smooth_mat.reshape(-1)


# RC transition matrix Q_rc

# Given current grid point r_grid[g], the AR(1) implies next-period RC is␣
 ↪normal with mean:
#   rho0_hat + rho1_hat * r_grid[g]
mu_rc = rho0_hat + rho1_hat * r_grid[:, None]

# Standardized upper bin edges for all current-state / next-bin combinations.
upper = (edges[1:][None, :] - mu_rc) / sigma_rho_hat
# Standardized lower bin edges for all current-state / next-bin combinations.
lower = (edges[:-1][None, :] - mu_rc) / sigma_rho_hat
Q_rc = ndtr(upper) - ndtr(lower)
Q_rc /= Q_rc.sum(axis=1, keepdims=True)

# Matrix-free transition operators
x_next_keep = np.array([1, 2, 3, 4, 5, 6, 6], dtype=np.int64)
x_reset = 0


# deterministic utility bases on the state grid
x_state_mat = np.repeat(np.arange(1, 8), K).reshape(7, K).astype(np.float64)
rc_state_mat = np.tile(r_grid, 7).reshape(7, K).astype(np.float64)

u0_base = -x_state_mat                  # maintenance cost base
u1_base = -rc_state_mat                 # replacement cost base

u0_base_vec = u0_base.reshape(-1)
u1_base_vec = u1_base.reshape(-1)
```

```python
# Build dense F0 and F1 once.
# Current state ordering is row-major on a 7 x K grid: state = x*K + g
F0 = np.zeros((S, S), dtype=np.float64)
F1 = np.zeros((S, S), dtype=np.float64)

for x in range(7):
    row_slice = slice(x * K, (x + 1) * K)

    keep_x = int(x_next_keep[x])
    col_slice_keep = slice(keep_x * K, (keep_x + 1) * K)
    F0[row_slice, col_slice_keep] = Q_rc

    col_slice_reset = slice(x_reset * K, (x_reset + 1) * K)
    F1[row_slice, col_slice_reset] = Q_rc

Fdiff = F1 - F0
I_S = np.eye(S, dtype=np.float64)

# two-action log-sum-exp and CCP map

def lse2(a, b):
    m = np.maximum(a, b)
    return m + np.log(np.exp(a - m) + np.exp(b - m))

def choice_prob_from_V_flat(theta1, theta2, V_flat):
    EV0 = F0 @ V_flat
    EV1 = F1 @ V_flat

    W0 = theta1 * u0_base_vec + beta * EV0
    W1 = theta2 * u1_base_vec + beta * EV1

    delta = W1 - W0
    p1 = expit(delta)

    return np.clip(p1, ccp_clip, 1.0 - ccp_clip), W0, W1, EV0, EV1

# NFXP Bellman solver with warm start

_nfxp_cache = {
    "eta": None,
    "V": np.zeros(S, dtype=np.float64),
}

def bellman_operator_nfxp(theta1, theta2, V_flat):
    EV0 = F0 @ V_flat
    EV1 = F1 @ V_flat
```

```
    W0 = theta1 * u0_base_vec + beta * EV0
    W1 = theta2 * u1_base_vec + beta * EV1

    return gamma_euler + lse2(W0, W1)

def solve_value_nfxp(theta1, theta2, tol=bellman_tol, maxiter=bellman_maxiter):
    V = _nfxp_cache["V"].copy()

    for it in range(maxiter):
        V_new = bellman_operator_nfxp(theta1, theta2, V)
        err = np.max(np.abs(V_new - V))
        V = V_new
        if err < tol:
            _nfxp_cache["V"] = V.copy()
            return V, it + 1

    _nfxp_cache["V"] = V.copy()
    return V, maxiter

# NFXP log-likelihood and exact gradient in eta-space
#    eta = log(theta), so theta = exp(eta) enforces positivity

def nfxp_objective_and_grad_eta(eta, inner_tol):
    theta = np.exp(eta)
    theta1 = float(theta[0])
    theta2 = float(theta[1])

    V, n_inner = solve_value_nfxp(theta1, theta2, tol=inner_tol)

    p1, W0, W1, EV0, EV1 = choice_prob_from_V_flat(theta1, theta2, V)
    ll = np.sum(N1_vec * np.log(p1) + N0_vec * np.log1p(-p1))

    p0 = 1.0 - p1

    # Bellman Jacobian wrt V:
    # D_V T = beta * [diag(p0) F0 + diag(p1) F1]
    DVT = beta * (p0[:, None] * F0 + p1[:, None] * F1)
    A = I_S - DVT

    # dT/dtheta_j
    dT_dtheta1 = p0 * u0_base_vec
    dT_dtheta2 = p1 * u1_base_vec

    # Implicit differentiation: (I - D_V T) dV/dtheta = dT/dtheta
    dV_dtheta1 = solve(A, dT_dtheta1, assume_a='gen', check_finite=False)
    dV_dtheta2 = solve(A, dT_dtheta2, assume_a='gen', check_finite=False)
```

```
    # delta = W1 - W0
    dDelta_dtheta1 = -u0_base_vec + beta * (Fdiff @ dV_dtheta1)
    dDelta_dtheta2 =  u1_base_vec + beta * (Fdiff @ dV_dtheta2)

    # For logit grouped likelihood:
    # d ell / d delta = N1 - N * p1
    score_weight = N1_vec - N_vec * p1
    score_theta1 = np.sum(score_weight * dDelta_dtheta1)
    score_theta2 = np.sum(score_weight * dDelta_dtheta2)

    # Chain rule: d ell / d eta_j = d ell / d theta_j * theta_j
    grad_eta = -np.array([
        score_theta1 * theta1,
        score_theta2 * theta2
    ], dtype=np.float64)

    return -ll, grad_eta, p1, V, n_inner

def nfxp_fun_stage1(eta):
    f, g, _, _, _ = nfxp_objective_and_grad_eta(eta,␣
 ↪inner_tol=bellman_tol_stage1)
    return f, g

def nfxp_fun_stage2(eta):
    f, g, _, _, _ = nfxp_objective_and_grad_eta(eta,␣
 ↪inner_tol=bellman_tol_stage2)
    return f, g

# HM direct solve and exact gradient in eta-space

p1_first_stage_vec = ccp_smooth_mat.reshape(-1)
p0_first_stage_vec = 1.0 - p1_first_stage_vec

neglog_p0_vec = -np.log(p0_first_stage_vec)
neglog_p1_vec = -np.log(p1_first_stage_vec)

# HM operator: V = b(theta) + beta * P_sigma V
P_sigma = p0_first_stage_vec[:, None] * F0 + p1_first_stage_vec[:, None] * F1
A_hm = I_S - beta * P_sigma
lu_hm, piv_hm = lu_factor(A_hm, check_finite=False)

# These derivatives are constant in theta because A_hm is fixed
dVhm_dtheta1_const = lu_solve((lu_hm, piv_hm), p0_first_stage_vec *␣
 ↪u0_base_vec, check_finite=False)
dVhm_dtheta2_const = lu_solve((lu_hm, piv_hm), p1_first_stage_vec *␣
 ↪u1_base_vec, check_finite=False)
```

```python
def solve_value_hm(theta1, theta2):
    u0 = theta1 * u0_base_vec
    u1 = theta2 * u1_base_vec

    b = (
        p0_first_stage_vec * (u0 + gamma_euler + neglog_p0_vec)
        + p1_first_stage_vec * (u1 + gamma_euler + neglog_p1_vec)
    )
    V = lu_solve((lu_hm, piv_hm), b, check_finite=False)
    return V

def hm_objective_and_grad_eta(eta):
    theta = np.exp(eta)
    theta1 = float(theta[0])
    theta2 = float(theta[1])

    V = solve_value_hm(theta1, theta2)

    p1, W0, W1, EV0, EV1 = choice_prob_from_V_flat(theta1, theta2, V)
    ll = np.sum(N1_vec * np.log(p1) + N0_vec * np.log1p(-p1))

    dDelta_dtheta1 = -u0_base_vec + beta * (Fdiff @ dVhm_dtheta1_const)
    dDelta_dtheta2 =  u1_base_vec + beta * (Fdiff @ dVhm_dtheta2_const)

    score_weight = N1_vec - N_vec * p1
    score_theta1 = np.sum(score_weight * dDelta_dtheta1)
    score_theta2 = np.sum(score_weight * dDelta_dtheta2)

    grad_eta = -np.array([
        score_theta1 * theta1,
        score_theta2 * theta2
    ], dtype=np.float64)

    return -ll, grad_eta, p1, V

def hm_fun(eta):
    f, g, _, _ = hm_objective_and_grad_eta(eta)
    return f, g

# Run estimators

eta0 = np.log(theta0)

# -------- NFXP: two-stage BFGS --------
t0 = time.perf_counter()

res_nfxp_stage1 = minimize(
```

```
    nfxp_fun_stage1,
    eta0,
    method="BFGS",
    jac=True,
    options={
        "gtol": 1e-4,
        "maxiter": 150,
        "disp": False,
    },
)

res_nfxp = minimize(
    nfxp_fun_stage2,
    res_nfxp_stage1.x,
    method="BFGS",
    jac=True,
    options={
        "gtol": 1e-4,
        "maxiter": 200,
        "disp": False,
    },
)

time_nfxp = time.perf_counter() - t0

theta_nfxp = np.exp(res_nfxp.x)
ll_nfxp, grad_nfxp, p1_nfxp, V_nfxp, nfxp_last_inner =␣
 ↪nfxp_objective_and_grad_eta(
    res_nfxp.x, inner_tol=bellman_tol_stage2
)
ll_nfxp = -ll_nfxp

# -------- HM: one-stage BFGS with exact gradient --------
t0 = time.perf_counter()

res_hm = minimize(
    hm_fun,
    eta0,
    method="BFGS",
    jac=True,
    options={
        "gtol": 1e-8,
        "maxiter": 200,
        "disp": False,
    },
)
```

```python
time_hm = time.perf_counter() - t0

theta_hm = np.exp(res_hm.x)
obj_hm, grad_hm, p1_hm, V_hm = hm_objective_and_grad_eta(res_hm.x)
ll_hm = -obj_hm

# Matrix-size report

dense_transition_shape = (S, S)
dense_transition_elements = S * S
dense_transition_memory_mb = dense_transition_elements * 8 / (1024 ** 2)

qrc_shape = Q_rc.shape
qrc_elements = Q_rc.size
qrc_memory_mb = qrc_elements * 8 / (1024 ** 2)

matrix_report = pd.DataFrame({
    "object": [
        "State space",
        "Dense F0/F1",
        "Stored RC transition Q_rc",
        "Smoothed CCP grid",
    ],
    "shape": [
        f"{S}",
        f"{dense_transition_shape}",
        f"{qrc_shape}",
        f"{ccp_smooth_mat.shape}",
    ],
    "elements": [
        S,
        dense_transition_elements,
        qrc_elements,
        ccp_smooth_mat.size,
    ],
    "approx_memory_MB": [
        np.nan,
        dense_transition_memory_mb,
        qrc_memory_mb,
        ccp_smooth_mat.size * 8 / (1024 ** 2),
    ],
})

# Final report

print("\n================ NFXP results ================")
print(f"success                 = {res_nfxp.success}")
```

```
print(f"message                 = {res_nfxp.message}")
print(f"loglike                 = {ll_nfxp:.6f}")
print(f"theta1_hat              = {theta_nfxp[0]:.6f}")
print(f"theta2_hat              = {theta_nfxp[1]:.6f}")
print(f"runtime_sec             = {time_nfxp:.4f}")
print(f"last_inner_iterations   = {nfxp_last_inner}")

print("\n============= Hotz-Miller results =============")
print(f"success                 = {res_hm.success}")
print(f"message                 = {res_hm.message}")
print(f"loglike                 = {ll_hm:.6f}")
print(f"theta1_hat              = {theta_hm[0]:.6f}")
print(f"theta2_hat              = {theta_hm[1]:.6f}")
print(f"runtime_sec             = {time_hm:.4f}")
print(f"HM value solve          = direct linear solve")

summary = pd.DataFrame({
    "method": ["NFXP", "Hotz-Miller MLE"],
    "theta1_hat": [theta_nfxp[0], theta_hm[0]],
    "theta2_hat": [theta_nfxp[1], theta_hm[1]],
    "rho0_hat": [rho0_hat, rho0_hat],
    "rho1_hat": [rho1_hat, rho1_hat],
    "sigma_rho_hat": [sigma_rho_hat, sigma_rho_hat],
    "loglike": [ll_nfxp, ll_hm],
    "runtime_sec": [time_nfxp, time_hm],
    "state_space_S": [S, S],
})

print("\n================ Summary table ================")
print(summary.to_string(index=False))

print("\n================ Matrix-size report ================")
print(matrix_report.to_string(index=False))

print("\nFirst 10 occupied-state empirical CCPs:")
print(empirical_ccp[occupied][:10])

print("\nFirst 10 occupied-state smoothed CCPs:")
print(ccp_smooth[occupied][:10])

print("\nFirst 10 occupied-state model CCPs from NFXP:")
print(p1_nfxp[occupied][:10])

print("\nFirst 10 occupied-state model CCPs from HM:")
print(p1_hm[occupied][:10])

print("\nRC grid midpoints:")
```

```python
print(r_grid)
```

```
Index(['i', 't', 'a', 'x', 'rc'], dtype='object')
First-stage estimates for RC_t:
rho0_hat      = 17.826900
rho1_hat      = 0.624909
sigma_rho_hat = 4.605933

Occupied states = 56 out of 56

================ NFXP results ================
success                = True
message                = Optimization terminated successfully.
loglike                = -33941.542312
theta1_hat             = 0.412523
theta2_hat             = 0.156883
runtime_sec            = 0.0822
last_inner_iterations  = 1

============= Hotz-Miller results =============
success                = True
message                = Optimization terminated successfully.
loglike                = -33941.896863
theta1_hat             = 0.411447
theta2_hat             = 0.156677
runtime_sec            = 0.0015
HM value solve         = direct linear solve

================ Summary table ================
         method  theta1_hat  theta2_hat  rho0_hat  rho1_hat  sigma_rho_hat
loglike  runtime_sec  state_space_S
           NFXP    0.412523    0.156883   17.8269  0.624909       4.605933
-33941.542312     0.082240             56
Hotz-Miller MLE    0.411447    0.156677   17.8269  0.624909       4.605933
-33941.896863     0.001535             56

================ Matrix-size report ================
                   object    shape  elements  approx_memory_MB
              State space       56        56               NaN
              Dense F0/F1 (56, 56)      3136          0.023926
Stored RC transition Q_rc   (8, 8)        64          0.000488
        Smoothed CCP grid   (7, 8)        56          0.000427

First 10 occupied-state empirical CCPs:
[0.02849003 0.01937658 0.00929487 0.00595238 0.00342173 0.00249377
 0.         0.         0.07763401 0.06640238]

First 10 occupied-state smoothed CCPs:
```

```
[0.03922644 0.02611197 0.01685497 0.01142194 0.0080005  0.00562819
 0.00369004 0.00374697 0.0976016  0.06556941]

First 10 occupied-state model CCPs from NFXP:
[0.02980554 0.01698012 0.00962913 0.00543595 0.00305523 0.00170994
 0.00095296 0.00052871 0.11062468 0.0682182 ]

First 10 occupied-state model CCPs from HM:
[0.02989062 0.0170364  0.00966707 0.00546186 0.00307282 0.00172161
 0.00096045 0.00053334 0.11093074 0.06837231]

RC grid midpoints:
[33.4375 37.3125 41.1875 45.0625 48.9375 52.8125 56.6875 60.5625]
```

1.5 Forward Simulation Method in Dynamic Estimation

1.5.1 (C. 1) Method: Forward simulation using estimated CCPs

Fix an initial state s and for simulation replication $r = 1, \dots, R$, generate a path $\{s_\tau^{(r)}, a_\tau^{(r)}\}_{\tau=0}^{T_{sim}}$ as follows:

First set $s_0^{(r)} = s$ and at each date τ:

1. Draw the action from the estimated CCP:

$$a_\tau^{(r)} \sim \text{Bernoulli}(\hat{P}(1 \mid s_\tau^{(r)})).$$

2. Given $a_\tau^{(r)}$, update mileage deterministically:

$$x_{\tau+1}^{(r)} = \begin{cases} 1, & a_\tau^{(r)} = 1 \\ \min(7, x_\tau^{(r)} + 1), & a_\tau^{(r)} = 0. \end{cases}$$

3. Update replacement cost using the AR(1) process as the parameters obtained in the reduced form part.

$$RC_{\tau+1}^{(r)} = \hat{\rho}_0 + \hat{\rho}_1 RC_\tau^{(r)} + \hat{\sigma}_\rho \eta_{\sigma+1}^{(r)}, \quad \eta_{\tau+1}^{(r)} \sim N(0,1)$$

Since RC is discrete, we can use the same method that we discussed in terms of the projection into the corresponding grids.

For the simulated path, define the truncation return:

$$G^{(1,r)}(s;\theta) = \sum_{\tau=0}^{T_{sim}} \beta^\tau [\bar{u}(s_\tau^{(r)}, a_\tau^{(r)}; \theta) + \gamma - \log \hat{P}(a_\tau^{(r)} \mid s_\tau^{(r)}].$$

where γ is again the Euler's constant. The term $\gamma - \log \hat{P}(a \mid s)$ is the conditional expectation of the selected Type-I extreme value shock under the CCP representation:

$$\mathbb{E}_a[\varepsilon_a \mid a \text{ chosen }, s] = \gamma - \log P(a \mid s)$$

Therefore, the simulated return in this expression is exactly the finite-horizon approximation to the value of following policy $\hat{P}$. This is the quantity prescribed by Eq. (5) in the assignment. Average across simulation replications to obtain the value estimator

$$\hat{V}^{(1)}(s;\theta) = \frac{1}{R}\sum_{r=1}^{R} G^{(1,r)}(s;\theta)$$

As $R \to \infty$, and $T_{sim} \to \infty$, this converges to the policy value $V^{\hat{P}}(s;\theta)$, provided the CCPs are bounded away from 0 and 1 and the transition process is correctly approximated.

Once $\hat{V}^{(1)}(s;\theta)$ has been computed for all states s the choice specific continuation values are obtained in the same way as:

$$\hat{EV}_a^{(1)}(s;\theta) = \sum_{s'} \hat{F}_a(s,s')\hat{V}^{(1)}(s',\theta)$$

Then the implied choice probabilities used in the likelihood are

$$\hat{p}^{(1)}(s;\theta) = \frac{\exp(-\theta_2 rc_s + \beta\hat{EV}_1^{(1)}(s;\theta))}{\exp(-\theta_1 x_s + \beta\hat{EV}_0^{(1)}(s;\theta)) + \exp(-\theta_2 rc_s + \beta\hat{EV}_1^{(1)}(s;\theta))}$$

and $\hat{p}_0^1(s;\theta) = 1 - \hat{p}_1^{(1)}(s;\theta)$. The corresponding state-level simulated log-likelihood is similarly constructed as:

$$\ell^{(1)}(\theta) = \sum_{s=1}^{S}[N_1(s)\log\hat{p}_1^{(1)}(s;\theta) + N_0(s)\log\hat{p}_0^{(1)}(s;\theta)]$$

So the estimator is

$$\hat{\theta}^{(1)} = \arg\max_{\theta} \ell^{(1)}(\theta)$$

This estimator is a two-step simulated pseudo-likelihood estimator: conditional on first-stage estimates $\hat{P}$ and $\hat{F}$, the continuation value is approximated by simulation rather than by fixed-point iteration.

1.5.2 (C. 2) Method: Forward simulation by drawing utility shocks directly

The second simulation method replaces the analytical correction term $\gamma - \log\hat{P}(a \mid s)$ by explicit simulation of the Type-I extreme value shocks. Again fix an initial state s, and for each simulation replication $r = 1, \ldots, R$, generate a path $\{s_\tau^{(r)}, a_\tau^{(r)}\}_{\tau=0}^{T_{sim}}$. Set

$$s_0^{(r)} = s.$$

At each draw τ, draw $\varepsilon_{0\tau}^{(r)}, \varepsilon_{1\tau}^{(r)}$ independently from the Type-I extreme value distribution. Then choose action according to the rule

$$a_\tau^{(r)} = 1 \iff \log \frac{\hat{P}(1 \mid s_\tau^{(r)})}{\hat{P}(0 \mid s_\tau^{(r)})} + \varepsilon_{1\tau}^{(r)} - \varepsilon_{0\tau}^{(r)} > 0.$$

This choice rule is valid because the difference of two independent Type-I extreme value shocks has the logistic distribution. Hence

$$\Pr(a = 1 \mid s) = \Pr(s_1 - s_0 > -\log \frac{\hat{P}(1 \mid s_\tau^{(r)})}{\hat{P}(0 \mid s_\tau^{(r)})} = \hat{P}(1 \mid s)$$

Therefore this procedure exactly reproduces the estimated CCP at each state. This is the content of Eq. (6) in the assignment.

Given the chosen action, update mileage and replacement cost in exactly the same way as in part (c.1). For the simulated path, we can correspondingly define the result as:

$$\hat{V}^{(2)}(s;\theta) = \frac{1}{R}\sum_{r=1}^{R} G^{(2,r)}(s;\theta).$$

where the truncated discounted return

$$G^{(2,r)}(s;\theta) = \sum_{\tau=0}^{T_{sim}} \beta^\tau [\bar{u}(s_\tau^{(r)}, a_\tau^{(r)};\theta) + \varepsilon_{a_\tau^{(r)},\tau}^{(r)}].$$

Once $\hat{V}^{(2)}(s;\theta)$ is computed for all states, we can correspondingly define:

$$\hat{EV}_a^{(2)}(s;\theta) = \sum_{s'} \hat{F}_a(s,s')\hat{V}^{(2)}(s',\theta)$$

Then the implied choice probabilities used in the likelihood are

$$\hat{p}^{(2)}(s;\theta) = \frac{\exp(-\theta_2 rc_s + \beta \hat{EV}_1^{(2)}(s;\theta))}{\exp(-\theta_1 x_s + \beta \hat{EV}_0^{(2)}(s;\theta)) + \exp(-\theta_2 rc_s + \beta \hat{EV}_1^{(2)}(s;\theta))}$$

and $\hat{p}_0^2(s;\theta) = 1 - \hat{p}_1^{(2)}(s;\theta)$. The corresponding state-level simulated log-likelihood is similarly constructed as:

$$\ell^{(2)}(\theta) = \sum_{s=1}^{S} [N_1(s)\log \hat{p}_1^{(2)}(s;\theta) + N_0(s)\log \hat{p}_0^{(2)}(s;\theta)]$$

So the estimator is

$$\hat{\theta}^{(2)} = \arg\max_{\theta} \ell^{(2)}(\theta)$$

1.5.3 Comparision between (C. 1) and (C. 2)$ methods:

The two simulation procedures approximate the same policy value $V^{\hat{P}}(s;\theta)$. The selected shock is integrated out analytically in the (C. 1)$, so the simulated path uses only the deterministic utility component plus this correction term. In part (C. 2), the shocks are drawn explicitly and the action is selected using the implied threshold rule. Thus part (C.1) generally has lower simulation variance, while part (C.2) is closer to the original random-utility representation of the model. Both are valid implementations of the forward-simulation estimator described in part (C).

The four step procedure can be considered as this:

1. Estimate transition matrix : exogenous (RC, empirical counts) + endogenous (mileage)
2. Estimate CCP $\hat{P}(a = j|s)$ from data (frequency estimator)
3. For each guess of θ: $V \leftarrow (\widehat{CCP}, \theta)$ via HM inversion; get P from V
4. Calculate likelihood given current θ

1.5.4 Notes for the Part (c)

The current estimation is using the heavily nested for loop and this is very slow for the python operation and I think to make the estimation faster, the best approach is to use C++ or parallel program or using the CUDA to optimize the operation. Apparently, we currently don't have enough time to optimize the code. But the overall structure of this code can still produce a consistent estimator.

Same as Part (a): we cut the observed RC range into N_{RC} equal-width bins and represent each by its midpoint. The transition matrix Π is built by counting empirical transitions in the data. This is fixed once and never recomputed during optimization, no parametric normal CDF, no re-evaluation at each optimizer call.

I'm using a baseline result in this block

[4]:
```
# Part (c): forward simulation estimators
#
# Key idea:
#
#   We exploit this by precomputing:
#     maint_component[s] = E_sim[sum beta^t * x_t * 1{a_t=0} | s0=s]
#     repl_component[s]  = E_sim[sum beta^t * rc_t * 1{a_t=1} | s0=s]
#     corr_component[s]  = E_sim[sum beta^t * (gamma - log P_hat(a_t|s_t)) |␣
↪s0=s]
#
#   for (c.1), and similarly:
#     shock_component[s] = E_sim[sum beta^t * eps_{a_t,t} | s0=s]
#
#   for (c.2).
#
#   Then:
```

```
#     V_hat^(1)(s;theta) = -theta1 * maint_c1[s] - theta2 * repl_c1[s] +␣
 ↪corr_c1[s]
#     V_hat^(2)(s;theta) = -theta1 * maint_c2[s] - theta2 * repl_c2[s] +␣
 ↪shock_c2[s]
#
#   So the optimizer no longer re-simulates the path.

# Simulation controls
Nsim_c1 = 100
Nsim_c2 = 200
Tsim = 100

sim_seed = 6622026
rng_sim = np.random.default_rng(sim_seed)

# State helpers and fixed first-stage objects
x_idx_of_state = np.repeat(np.arange(7, dtype=np.int16), K)
g_idx_of_state = np.tile(np.arange(K, dtype=np.int16), 7)

# Initial state matrices for vectorized simulation:
# row s = initial state s, column r = replication r
x0_c1 = np.repeat(x_idx_of_state[:, None], Nsim_c1, axis=1)
g0_c1 = np.repeat(g_idx_of_state[:, None], Nsim_c1, axis=1)

x0_c2 = np.repeat(x_idx_of_state[:, None], Nsim_c2, axis=1)
g0_c2 = np.repeat(g_idx_of_state[:, None], Nsim_c2, axis=1)

# Fixed first-stage CCP from part (b)
p1_policy = ccp_smooth.copy()
p0_policy = 1.0 - p1_policy
logit_policy = np.log(p1_policy) - np.log(p0_policy)

# RC-grid transition CDF for inverse-CDF simulation
Q_rc_cdf = np.cumsum(Q_rc, axis=1)
Q_rc_cdf[:, -1] = 1.0

# Positive x and rc values on the state grid
x_pos_vec = -u0_base_vec.copy()   # because u0_base_vec = -x
rc_pos_vec = -u1_base_vec.copy()  # because u1_base_vec = -rc

# Common random numbers

# (c.1): action uniforms and RC uniforms
u_action_c1 = rng_sim.random((S, Nsim_c1, Tsim + 1), dtype=np.float64)

half_c1 = Nsim_c1 // 2
u_rc_half_c1 = rng_sim.random((S, half_c1, Tsim), dtype=np.float64)
```

```
u_rc_c1 = np.concatenate([u_rc_half_c1, 1.0 - u_rc_half_c1], axis=1)
if u_rc_c1.shape[1] < Nsim_c1:
    extra = rng_sim.random((S, 1, Tsim), dtype=np.float64)
    u_rc_c1 = np.concatenate([u_rc_c1, extra], axis=1)

# (c.2): explicit shocks and RC uniforms
half_c2 = Nsim_c2 // 2

u0_half = rng_sim.random((S, half_c2, Tsim + 1), dtype=np.float64)
u1_half = rng_sim.random((S, half_c2, Tsim + 1), dtype=np.float64)

u0_full = np.concatenate([u0_half, 1.0 - u0_half], axis=1)
u1_full = np.concatenate([u1_half, 1.0 - u1_half], axis=1)

u0_full = np.clip(u0_full, 1e-12, 1.0 - 1e-12)
u1_full = np.clip(u1_full, 1e-12, 1.0 - 1e-12)

eps0_c2 = -np.log(-np.log(u0_full))
eps1_c2 = -np.log(-np.log(u1_full))

u_rc_half_c2 = rng_sim.random((S, half_c2, Tsim), dtype=np.float64)
u_rc_c2 = np.concatenate([u_rc_half_c2, 1.0 - u_rc_half_c2], axis=1)
if u_rc_c2.shape[1] < Nsim_c2:
    extra = rng_sim.random((S, 1, Tsim), dtype=np.float64)
    u_rc_c2 = np.concatenate([u_rc_c2, extra], axis=1)

# Vectorized RC-grid transition sampler
def draw_next_rc_grid_mat(g_idx_mat, u_mat):
    cdf_rows = Q_rc_cdf[g_idx_mat]
    g_next = np.sum(u_mat[:, :, None] > cdf_rows, axis=2)
    return np.clip(g_next, 0, K - 1).astype(np.int16)

# Precompute simulation components for (c.1)
def precompute_components_c1():
    x_idx = x0_c1.copy()
    g_idx = g0_c1.copy()

    maint_acc = np.zeros((S, Nsim_c1), dtype=np.float64)
    repl_acc  = np.zeros((S, Nsim_c1), dtype=np.float64)
    corr_acc  = np.zeros((S, Nsim_c1), dtype=np.float64)

    discount = 1.0

    for tau in range(Tsim + 1):
        state_now = x_idx * K + g_idx

        p1_now = p1_policy[state_now]
```

```
        a1 = (u_action_c1[:, :, tau] < p1_now)
        a0 = ~a1

        p_selected = np.where(a1, p1_now, 1.0 - p1_now)

        maint_acc += discount * x_pos_vec[state_now] * a0
        repl_acc  += discount * rc_pos_vec[state_now] * a1
        corr_acc  += discount * (gamma_euler - np.log(p_selected))

        if tau < Tsim:
            x_idx = np.where(a1, 0, x_next_keep[x_idx]).astype(np.int16)
            g_idx = draw_next_rc_grid_mat(g_idx, u_rc_c1[:, :, tau])
            discount *= beta

    maint_component = maint_acc.mean(axis=1)
    repl_component  = repl_acc.mean(axis=1)
    corr_component  = corr_acc.mean(axis=1)

    return maint_component, repl_component, corr_component

# Precompute simulation components for (c.2)
def precompute_components_c2():
    x_idx = x0_c2.copy()
    g_idx = g0_c2.copy()

    maint_acc = np.zeros((S, Nsim_c2), dtype=np.float64)
    repl_acc  = np.zeros((S, Nsim_c2), dtype=np.float64)
    shock_acc = np.zeros((S, Nsim_c2), dtype=np.float64)

    discount = 1.0

    for tau in range(Tsim + 1):
        state_now = x_idx * K + g_idx

        logit_now = logit_policy[state_now]
        e0_now = eps0_c2[:, :, tau]
        e1_now = eps1_c2[:, :, tau]

        a1 = (logit_now + e1_now - e0_now > 0.0)
        a0 = ~a1

        eps_selected = np.where(a1, e1_now, e0_now)

        maint_acc += discount * x_pos_vec[state_now] * a0
        repl_acc  += discount * rc_pos_vec[state_now] * a1
        shock_acc += discount * eps_selected
```

```
        if tau < Tsim:
            x_idx = np.where(a1, 0, x_next_keep[x_idx]).astype(np.int16)
            g_idx = draw_next_rc_grid_mat(g_idx, u_rc_c2[:, :, tau])
            discount *= beta

    maint_component = maint_acc.mean(axis=1)
    repl_component  = repl_acc.mean(axis=1)
    shock_component = shock_acc.mean(axis=1)

    return maint_component, repl_component, shock_component

# One-time precomputation
t0 = time.perf_counter()
maint_c1, repl_c1, corr_c1 = precompute_components_c1()
time_precomp_c1 = time.perf_counter() - t0

t0 = time.perf_counter()
maint_c2, repl_c2, shock_c2 = precompute_components_c2()
time_precomp_c2 = time.perf_counter() - t0

print("\nPrecomputation for revised part (c):")
print(f"time_precomp_c1        = {time_precomp_c1:.4f}")
print(f"time_precomp_c2        = {time_precomp_c2:.4f}")

# Theta-to-value maps using precomputed components
def simulated_value_c1(theta1, theta2):
    return -theta1 * maint_c1 - theta2 * repl_c1 + corr_c1

def simulated_value_c2(theta1, theta2):
    return -theta1 * maint_c2 - theta2 * repl_c2 + shock_c2

# Precompute exact derivatives of delta with respect to theta
#
# Since:
#   V^(1)(theta) = -theta1 * maint_c1 - theta2 * repl_c1 + corr_c1
# we have:
#   dV^(1)/dtheta1 = -maint_c1
#   dV^(1)/dtheta2 = -repl_c1
#
# and similarly for (c.2).
dDelta_c1_dtheta1 = -u0_base_vec - beta * (Fdiff @ maint_c1)
dDelta_c1_dtheta2 =  u1_base_vec - beta * (Fdiff @ repl_c1)

dDelta_c2_dtheta1 = -u0_base_vec - beta * (Fdiff @ maint_c2)
dDelta_c2_dtheta2 =  u1_base_vec - beta * (Fdiff @ repl_c2)

# Exact objective and gradient for (c.1) and (c.2)
```

```python
def sim_c1_objective_and_grad_eta(eta):
    theta = np.exp(eta)
    theta1 = float(theta[0])
    theta2 = float(theta[1])

    V_sim = simulated_value_c1(theta1, theta2)
    p1_model, _, _, _, _ = choice_prob_from_V_flat(theta1, theta2, V_sim)

    ll = np.sum(N1_vec * np.log(p1_model) + N0_vec * np.log1p(-p1_model))

    score_weight = N1_vec - N_vec * p1_model
    score_theta1 = np.sum(score_weight * dDelta_c1_dtheta1)
    score_theta2 = np.sum(score_weight * dDelta_c1_dtheta2)

    grad_eta = -np.array([
        score_theta1 * theta1,
        score_theta2 * theta2,
    ], dtype=np.float64)

    return -ll, grad_eta, p1_model, V_sim

def sim_c2_objective_and_grad_eta(eta):
    theta = np.exp(eta)
    theta1 = float(theta[0])
    theta2 = float(theta[1])

    V_sim = simulated_value_c2(theta1, theta2)
    p1_model, _, _, _, _ = choice_prob_from_V_flat(theta1, theta2, V_sim)

    ll = np.sum(N1_vec * np.log(p1_model) + N0_vec * np.log1p(-p1_model))

    score_weight = N1_vec - N_vec * p1_model
    score_theta1 = np.sum(score_weight * dDelta_c2_dtheta1)
    score_theta2 = np.sum(score_weight * dDelta_c2_dtheta2)

    grad_eta = -np.array([
        score_theta1 * theta1,
        score_theta2 * theta2,
    ], dtype=np.float64)

    return -ll, grad_eta, p1_model, V_sim

def sim_c1_fun(eta):
    f, g, _, _ = sim_c1_objective_and_grad_eta(eta)
    return f, g

def sim_c2_fun(eta):
```

```
    f, g, _, _ = sim_c2_objective_and_grad_eta(eta)
    return f, g

# Estimate (c.1)
#     Start from HM, as before, but now no repeated simulation.
eta_start_c1 = res_hm.x.copy()

t0 = time.perf_counter()
res_c1 = minimize(
    sim_c1_fun,
    eta_start_c1,
    method="BFGS",
    jac=True,
    options={
        "gtol": 1e-6,
        "maxiter": 200,
        "disp": False,
    },
)
time_opt_c1 = time.perf_counter() - t0
time_c1_total = time_precomp_c1 + time_opt_c1

theta_c1 = np.exp(res_c1.x)
obj_c1, grad_c1, p1_c1, V_c1 = sim_c1_objective_and_grad_eta(res_c1.x)
ll_c1 = -obj_c1

# Estimate (c.2)
#     Start from (c.1), as before, but now no repeated simulation.
eta_start_c2 = res_c1.x.copy()

t0 = time.perf_counter()
res_c2 = minimize(
    sim_c2_fun,
    eta_start_c2,
    method="BFGS",
    jac=True,
    options={
        "gtol": 1e-6,
        "maxiter": 200,
        "disp": False,
    },
)
time_opt_c2 = time.perf_counter() - t0
time_c2_total = time_precomp_c2 + time_opt_c2

theta_c2 = np.exp(res_c2.x)
obj_c2, grad_c2, p1_c2, V_c2 = sim_c2_objective_and_grad_eta(res_c2.x)
```

```python
ll_c2 = -obj_c2

# Reports
print("\n============= Revised forward simulation (c.1) results =============")
print(f"success                 = {res_c1.success}")
print(f"message                 = {res_c1.message}")
print(f"loglike                 = {ll_c1:.6f}")
print(f"theta1_hat              = {theta_c1[0]:.6f}")
print(f"theta2_hat              = {theta_c1[1]:.6f}")
print(f"time_precompute_sec     = {time_precomp_c1:.4f}")
print(f"time_optimize_sec       = {time_opt_c1:.4f}")
print(f"runtime_sec_total       = {time_c1_total:.4f}")
print(f"Nsim                    = {Nsim_c1}")
print(f"Tsim                    = {Tsim}")

print("\n============= Revised forward simulation (c.2) results =============")
print(f"success                 = {res_c2.success}")
print(f"message                 = {res_c2.message}")
print(f"loglike                 = {ll_c2:.6f}")
print(f"theta1_hat              = {theta_c2[0]:.6f}")
print(f"theta2_hat              = {theta_c2[1]:.6f}")
print(f"time_precompute_sec     = {time_precomp_c2:.4f}")
print(f"time_optimize_sec       = {time_opt_c2:.4f}")
print(f"runtime_sec_total       = {time_c2_total:.4f}")
print(f"Nsim                    = {Nsim_c2}")
print(f"Tsim                    = {Tsim}")

# Append revised part (c) results to summary_all
summary_part_c = pd.DataFrame({
    "method": ["Forward simulation (c.1)", "Forward simulation (c.2)"],
    "theta1_hat": [theta_c1[0], theta_c2[0]],
    "theta2_hat": [theta_c1[1], theta_c2[1]],
    "rho0_hat": [rho0_hat, rho0_hat],
    "rho1_hat": [rho1_hat, rho1_hat],
    "sigma_rho_hat": [sigma_rho_hat, sigma_rho_hat],
    "loglike": [ll_c1, ll_c2],
    "runtime_sec": [time_c1_total, time_c2_total],
    "state_space_S": [S, S],
})

summary_all = pd.concat([summary, summary_part_c], ignore_index=True)

print("\n================ Updated summary table with revised part (c)␣
↪================")
print(summary_all.to_string(index=False))
```

```python
print("\nFirst 10 occupied-state model CCPs from revised forward simulation (c.
↪1):")
print(p1_c1[occupied][:10])

print("\nFirst 10 occupied-state model CCPs from revised forward simulation (c.
↪2):")
print(p1_c2[occupied][:10])
```

```
Precomputation for revised part (c):
time_precomp_c1        = 0.0389
time_precomp_c2        = 0.0880

============= Revised forward simulation (c.1) results =============
success                = True
message                = Optimization terminated successfully.
loglike                = -33943.367452
theta1_hat             = 0.408617
theta2_hat             = 0.155240
time_precompute_sec    = 0.0389
time_optimize_sec      = 0.0012
runtime_sec_total      = 0.0401
Nsim                   = 100
Tsim                   = 100

============= Revised forward simulation (c.2) results =============
success                = True
message                = Optimization terminated successfully.
loglike                = -34007.929680
theta1_hat             = 0.411057
theta2_hat             = 0.158263
time_precompute_sec    = 0.0880
time_optimize_sec      = 0.0008
runtime_sec_total      = 0.0888
Nsim                   = 200
Tsim                   = 100

================ Updated summary table with revised part (c) ================
                  method  theta1_hat  theta2_hat  rho0_hat  rho1_hat
sigma_rho_hat       loglike  runtime_sec  state_space_S
                    NFXP    0.412523    0.156883   17.8269  0.624909
4.605933 -33941.542312     0.082240             56
         Hotz-Miller MLE    0.411447    0.156677   17.8269  0.624909
4.605933 -33941.896863     0.001535             56
Forward simulation (c.1)    0.408617    0.155240   17.8269  0.624909
4.605933 -33943.367452     0.040132             56
Forward simulation (c.2)    0.411057    0.158263   17.8269  0.624909
```

```
4.605933 -34007.929680     0.088821             56

First 10 occupied-state model CCPs from revised forward simulation (c.1):
[0.0316883  0.01787518 0.01005004 0.00564801 0.00318087 0.00179285
 0.00100739 0.0005632  0.11331042 0.06924616]

First 10 occupied-state model CCPs from revised forward simulation (c.2):
[0.03131684 0.01877038 0.0109925  0.00617823 0.00329412 0.00167239
 0.00082738 0.00041206 0.13584602 0.08537591]
```

1.6 Regression Based Estimation without the Dynamic Setup

To make the notation simplier, we denote, for each date t,

$$p_{1,t} = \Pr(a_t = 1 \mid x_t = x, RC_t), \quad p_{0,t}(x) = 1 - p_{1,t}(x)$$

Since RC_t is constant within month t, there are one dimensional CCPs in x for each t. Let $m(x) = \min(x+1, 7)$.

Start from the logit choice-probability identity. Under the tyle -I extreme value shocks, we have:

$$\log \frac{p_{0,t}(x)}{p_{1,t}(x)} = [-\theta_1 x + \beta \mathbb{E}_t V_{t+1}(m(x), RC_{t+1})] - [-\theta_2 RC_t + \beta \mathbb{E}_t[V_{t+1}(1, RC_{t+1})].$$

Rearrange and we have:

$$\log \frac{p_{0,t}(x)}{p_{1,t}(x)} = -\theta_1 x + \theta_2 RC_t + \beta \mathbb{E}_t[V_{t+1}(m(x), RC_{t+1}) - V_{t+1}(1, RC_{t+1})].$$

ow use the logit identity for the next period. For any next-period state (x', RC_{t+1}):

$$V_{t+1}(x', RC_{t+1}) = v_{1,t+1}(x', RC_{t+1}) - \log p_{1,t+1}(x') + \gamma$$

where $v_{1,t+1}(x', RC_{t+1}) = -\theta_2 RC_{t+1} + \beta \mathbb{E}_t[V_{t+2}(1, RC_{t+2}) \mid RC_{t+1}]$. Crucially, $v_{1,t+1}(x', RC_{t+1})$ does not depend on x' because if action 1 is chosen at time $t+1$ then mileage is always reset to 1 and the future law of RC is exogenous and independent on the current mileage. Therefore, when we difference two next-period states with the same RC_{t+1} but different mileage, the common terms cancel:

$$V_{t+1}(m(x), RC_{t+1}) - V_{t+1}(1, RC_{t+1}) = \log \frac{p_{1,t+1}(1)}{p_{1,t+1}(m(x))}$$

Substitute the above tow equations. This gives the exact Scott-style linear relation:

$$\log \frac{p_{0,t}(x)}{p_{1,t}(x)} + \beta \mathbb{E}_t[\log \frac{p_{0,t}(m(x))}{p_{1,t}(1)}] = -\theta_1 x + \theta_2 RC + \varepsilon_1 + \varepsilon_2$$

To implement the above equation, we discretize RC into $K_d = 5$ bins. Let the bins be indexed by $g = 1, \dots, 5$ and let rc_g denotes the midpoint of bin g. The current state is then (x, g) with $x \in \{1, \dots, 7\}$. After estimation of the $(\hat{\rho}_0, \hat{\rho}_1, \hat{\sigma}_\rho)$. Then we can denote the frequency CCP as:

$$\tilde{p}_1(x, g) = \frac{N_1(x, g)}{N_2(x, g)}$$

whenever $N(x, g) > 0$. To remove some states that are sparse and frequency etimates may hit 0 or 1. Accordingly, we estimate a parametric logit model on the (x, g) state space:

$$\Pr(a = 1 \mid x, g) = \Lambda(z(x, rc_g)^\top \alpha), \quad \Lambda(u) = \frac{e^u}{1 + e^u}$$

where the regressor vector is the third-degree polynomial basis

$$z(x, rc_g) = (1, x, rc_g, x^2, rc_g^2, xrc_g, x^3, rc_g^3, x^2 rc_g, xrc_g^2)$$

The smooth CCP is then

$$\hat{p}_1(x, g) = \Lambda(z(x, rc_g)^\top \hat{\alpha}), \quad \hat{p}_0(x, g) = 1 - \hat{p}_1(x, g)$$

The weight assigned to row (x, g, h) can be constructed as

$$w(x, g, h) = N(x, g) Q(g, h)$$

Then we can implement the weighted least squares estimation.

To derive the estimator we can write:

$\hat{\Lambda}_t(x) = \log \frac{1 - \hat{p}_{1,t}(x)}{\hat{p}_{1,t}(x)}$ and because the dynamic term requires $t + 1$, define the realized continuation proxy $\hat{C}_t(x) = \log \frac{\hat{p}_{1,t+1}(m(x))}{\hat{p}_{1,t+1}(1)}$. Then the regression dependent variable is

$$\hat{Y}_t(x) = \hat{\Lambda}_t(x) + \hat{C}_t(x) + u_t(x)$$

where the residual $u_t(x)$ contains first-stage estimation error and the difference between the realized next-period continuation term and its conditional expectation. Under rational expectations and exogeneity of RC_t,

$$\mathbb{E}_t[u_t(x) \mid x, RC_t] = 0$$

So the estimator can be written as weighted least squares over (t, x_t) with $w_t = n_t(x)$ in the setup:

$$\hat{\theta}^{reg} = \arg\min_{\theta_1, \theta_2} \sum_{x=1}^{7} \sum_{g=1}^{5} \sum_{h=1}^{5} w(x, g, h)[Y(x, g, h) - \theta_x x - \theta_{rc} rc_g]^2$$

we can also denote the matrix notations

$$X = \begin{pmatrix} x_1 & rc_1 \\ x_2 & rc_2 \\ \dots & \dots \\ x_R & rc_R \end{pmatrix} \quad W = \text{diag}(w_1, \dots, w_R),$$

where each r corresponds to one triple (x, g, h). Then the WLs estimator is:

$$\hat{\theta} = (X^\top W X)^{-1} X^\top W Y$$

In terms of the part (d) I estiamte a model that is more consistent with the set up of the solution using a polynomial approximation since the grid's smoothing average may not be a well approximated results.

[5]:
```
#   1. estimate reduced-form RC transition by OLS
#   2. discretize RC into 5 grids
#   3. estimate CCP p1(x,g) on the (x, RC-grid) state space
#      using a parametric 3rd-degree logit smoother
#   4. build 5 OLS rows per current state (x,g), one for each
#      possible next-period RC grid h
#   5. run weighted least squares with weight N(x,g)*Q(g,h)

beta = 0.95
K_d = 5
ccp_clip = 1e-10

# ------------------------------------------------------------
# Helper: polynomial features for parametric CCP smoothing
# a parametric logit using third-degree polynomials in x and RC.
# ------------------------------------------------------------
def poly3_features(x, rc):
    x = np.asarray(x, dtype=np.float64)
    rc = np.asarray(rc, dtype=np.float64)

    return np.column_stack([
        np.ones_like(x),
        x,
        rc,
        x**2,
        rc**2,
        x * rc,
        x**3,
        rc**3,
        (x**2) * rc,
        x * (rc**2),
    ])
```

```python
t0_total = time.perf_counter()

t0 = time.perf_counter()

Y = rc_path[1:]
X = np.column_stack([np.ones(T - 1), rc_path[:-1]])

rho_ols = np.linalg.lstsq(X, Y, rcond=None)[0]
rho0_hat = float(rho_ols[0])
rho1_hat = float(np.clip(rho_ols[1], -0.999, 0.999))

resid = Y - X @ rho_ols
sigma_rho_hat = float(np.sqrt(np.mean(resid**2)))
sigma_rho_hat = max(sigma_rho_hat, 1e-12)

time_rc = time.perf_counter() - t0

print("\nReduced-form RC estimates:")
print(f"rho0_hat               = {rho0_hat:.6f}")
print(f"rho1_hat               = {rho1_hat:.6f}")
print(f"sigma_rho_hat          = {sigma_rho_hat:.6f}")
print(f"runtime_sec            = {time_rc:.4f}")

# Discretize RC into 5 grids for part (d)
#    The sample solution explicitly uses 5 grids in part (d).
rc_min = float(rc_path.min())
rc_max = float(rc_path.max())

edges_d = np.linspace(rc_min, rc_max, K_d + 1)
r_grid_d = 0.5 * (edges_d[:-1] + edges_d[1:])

# Month-level RC bin assignment
rc_bin_by_month_d = np.searchsorted(edges_d, rc_path, side="right") - 1
rc_bin_by_month_d = np.clip(rc_bin_by_month_d, 0, K_d - 1).astype(np.int16)

# Align month-level bins with flattened bus-month observations
rc_bin_flat_d = np.tile(rc_bin_by_month_d, N)

print("\nPart (d) RC grid midpoints:")
print(r_grid_d)

# State-level counts on (x, g)
#    State id = x_index * K_d + g_index
S_d = 7 * K_d
state_id_d = x_flat * K_d + rc_bin_flat_d
```

```
N1_vec_d = np.bincount(state_id_d[a_flat == 1], minlength=S_d).astype(np.
 ↪float64)
N0_vec_d = np.bincount(state_id_d[a_flat == 0], minlength=S_d).astype(np.
 ↪float64)
N_vec_d  = N1_vec_d + N0_vec_d

occupied_d = N_vec_d > 0

N1_mat_d = N1_vec_d.reshape(7, K_d)
N0_mat_d = N0_vec_d.reshape(7, K_d)
N_mat_d  = N_vec_d.reshape(7, K_d)

print("\nState-count summary for part (d):")
print(f"Total states            = {S_d}")
print(f"Occupied states         = {int(occupied_d.sum())} out of {S_d}")

# Parametric CCP smoothing on (x, g)

t0 = time.perf_counter()

# State grid values
x_state_mat_d = np.repeat(np.arange(1, 8), K_d).reshape(7, K_d).astype(np.
 ↪float64)
rc_state_mat_d = np.tile(r_grid_d, 7).reshape(7, K_d).astype(np.float64)

x_state_vec_d = x_state_mat_d.reshape(-1)
rc_state_vec_d = rc_state_mat_d.reshape(-1)

# Keep only occupied states in fitting
mask_fit = occupied_d.copy()

x_fit = x_state_vec_d[mask_fit]
rc_fit = rc_state_vec_d[mask_fit]

N1_fit = N1_vec_d[mask_fit]
N0_fit = N0_vec_d[mask_fit]

# Polynomial design matrix on occupied states
Z_fit = poly3_features(x_fit, rc_fit)

# Build a weighted binary-response dataset:
# one "success" row with weight N1, one "failure" row with weight N0
X_logit = np.vstack([Z_fit, Z_fit])
y_logit = np.concatenate([
    np.ones_like(N1_fit, dtype=np.int8),
    np.zeros_like(N0_fit, dtype=np.int8),
])
```

```python
w_logit = np.concatenate([N1_fit, N0_fit])

# Fit logistic regression with essentially no regularization
# fit_intercept=False because the first column of Z_fit is already 1
logit_model = LogisticRegression(
    penalty="l2",
    C=1e6,
    fit_intercept=False,
    solver="lbfgs",
    max_iter=10000,
)

logit_model.fit(X_logit, y_logit, sample_weight=w_logit)

# Predict smoothed CCPs on all 35 states
Z_all = poly3_features(x_state_vec_d, rc_state_vec_d)
p1_hat_d = logit_model.predict_proba(Z_all)[:, 1]
p1_hat_d = np.clip(p1_hat_d, ccp_clip, 1.0 - ccp_clip)

p0_hat_d = 1.0 - p1_hat_d

p1_hat_d_mat = p1_hat_d.reshape(7, K_d)
p0_hat_d_mat = p0_hat_d.reshape(7, K_d)

time_ccp = time.perf_counter() - t0

print("\nParametric CCP smoothing results:")
print(f"runtime_sec            = {time_ccp:.4f}")
print("First 10 occupied-state smoothed CCPs:")
print(p1_hat_d[occupied_d][:10])

# 5-grid transition matrix Q(g,h) for RC
#    Current grid representative value is r_grid_d[g].

mu_rc_d = rho0_hat + rho1_hat * r_grid_d[:, None]

upper_d = (edges_d[1:][None, :] - mu_rc_d) / sigma_rho_hat
lower_d = (edges_d[:-1][None, :] - mu_rc_d) / sigma_rho_hat

Q_rc_d = norm.cdf(upper_d) - norm.cdf(lower_d)
Q_rc_d /= Q_rc_d.sum(axis=1, keepdims=True)

print("\nFirst row of Q_rc_d:")
print(Q_rc_d[0])
```

```
#    For each current state (x,g), create K_d rows, one for each
#    possible next-period RC grid h.
#
#    Current-state log-odds:
#      Lambda(x,g) = log[p0(x,g) / p1(x,g)]
#
#    Continuation piece for row (x,g,h):
#      log[p1(m(x),h) / p1(1,h)]
#
#    Regression row:
#      Y(x,g,h) = Lambda(x,g) + beta * log[p1(m(x),h) / p1(1,h)]
#
#    WLS weight:
#      w(x,g,h) = N(x,g) * Q(g,h)

t0 = time.perf_counter()

Lambda_d_mat = np.log(p0_hat_d_mat / p1_hat_d_mat)

# m(x) = min(x+1, 7) in 0-based indexing
m_idx = np.array([1, 2, 3, 4, 5, 6, 6], dtype=np.int64)

rows_y = []
rows_x = []
rows_rc = []
rows_w = []

for x0 in range(7):        # current mileage index
    x_val = float(x0 + 1)
    mx = int(m_idx[x0])

    for g in range(K_d):   # current RC grid
        n_xg = N_mat_d[x0, g]

        if n_xg <= 0:
            continue

        lam_xg = Lambda_d_mat[x0, g]
        rc_g = float(r_grid_d[g])

        for h in range(K_d):   # next-period RC grid
            w_xgh = n_xg * Q_rc_d[g, h]

            cont_xgh = np.log(p1_hat_d_mat[mx, h] / p1_hat_d_mat[0, h])
            y_xgh = lam_xg + beta * cont_xgh

            rows_y.append(y_xgh)
```

```
            rows_x.append(x_val)
            rows_rc.append(rc_g)
            rows_w.append(w_xgh)

y_reg = np.asarray(rows_y, dtype=np.float64)
x_reg = np.asarray(rows_x, dtype=np.float64)
rc_reg = np.asarray(rows_rc, dtype=np.float64)
w_reg = np.asarray(rows_w, dtype=np.float64)

# The professor-style state-level OLS should have:
# (# occupied current states) * K_d rows
expected_rows = int(occupied_d.sum()) * K_d

print("\nOLS row-construction check:")
print(f"num_reg_rows            = {len(y_reg)}")
print(f"expected_reg_rows       = {expected_rows}")
print(f"total_weight            = {w_reg.sum():.0f}")

# Weighted least squares without intercept
#
#    Theory:
#      Y(x,g,h) = -theta1 * x + theta2 * RC_g + u(x,g,h)
#
#    So if regression is:
#      Y = b_x * x + b_rc * RC
#    then:
#      theta1 = -b_x
#      theta2 =  b_rc

X_reg = np.column_stack([x_reg, rc_reg])

sqrt_w = np.sqrt(w_reg)
Xw = X_reg * sqrt_w[:, None]
yw = y_reg * sqrt_w

coef_reg = np.linalg.lstsq(Xw, yw, rcond=None)[0]

b_x_hat = float(coef_reg[0])
b_rc_hat = float(coef_reg[1])

theta1_reg = -b_x_hat
theta2_reg =  b_rc_hat

# Fit diagnostics

yhat_reg = X_reg @ coef_reg
resid_reg = y_reg - yhat_reg
```

```python
sse_w = float(np.sum(w_reg * resid_reg**2))
rmse_w = float(np.sqrt(sse_w / np.sum(w_reg)))

sst0_w = float(np.sum(w_reg * y_reg**2))
r2_uncentered_w = 1.0 - sse_w / sst0_w if sst0_w > 0 else np.nan

time_reg = time.perf_counter() - t0

# Final report
time_total = time.perf_counter() - t0_total

print("\n============= Linear regression (d) results =============")
print(f"theta1_hat             = {theta1_reg:.6f}")
print(f"theta2_hat             = {theta2_reg:.6f}")
print(f"coef_on_x              = {b_x_hat:.6f}")
print(f"coef_on_rc             = {b_rc_hat:.6f}")
print(f"runtime_sec_reg        = {time_reg:.4f}")
print(f"runtime_sec_total      = {time_total:.4f}")
print(f"weighted_rmse          = {rmse_w:.6f}")
print(f"weighted_R2_uncentered = {r2_uncentered_w:.6f}")

print("\nFirst 10 OLS dependent-variable rows Y(x,g,h):")
print(y_reg[:10])

# ------------------------------------------------------------
# Append part (d) to the existing summary table
# ------------------------------------------------------------

summary_part_d = pd.DataFrame({
    "method": ["Linear regression (reduced-form WLS)"],
    "theta1_hat": [theta1_reg],
    "theta2_hat": [theta2_reg],
    "rho0_hat": [rho0_hat],
    "rho1_hat": [rho1_hat],
    "sigma_rho_hat": [sigma_rho_hat],
    "loglike": [np.nan],          # part (d) is not MLE
    "runtime_sec": [time_total],  # use total runtime for part (d)
    "state_space_S": [S_d],       # 7 * 5 = 35 for this implementation
})

if "summary_all" in globals():
    summary_all = pd.concat([summary_all, summary_part_d], ignore_index=True)
elif "summary" in globals():
    summary_all = pd.concat([summary, summary_part_d], ignore_index=True)
else:
    summary_all = summary_part_d.copy()
```

```python
print("\n================ Final summary table ================")
print(summary_all.to_string(index=False))
```

```
Reduced-form RC estimates:
rho0_hat                = 17.826900
rho1_hat                = 0.624909
sigma_rho_hat           = 4.605933
runtime_sec             = 0.0007

Part (d) RC grid midpoints:
[34.6 40.8 47.  53.2 59.4]

State-count summary for part (d):
Total states             = 35
Occupied states          = 35 out of 35

Parametric CCP smoothing results:
runtime_sec             = 0.0176
First 10 occupied-state smoothed CCPs:
[0.02962737 0.01139535 0.00434986 0.00173373 0.00075591 0.09059053
 0.042051   0.018714   0.00846126 0.00409174]

First row of Q_rc_d:
[3.23549980e-01 5.02177980e-01 1.63440263e-01 1.06996816e-02
 1.32096374e-04]

OLS row-construction check:
num_reg_rows             = 175
expected_reg_rows        = 175
total_weight             = 100000

============= Linear regression (d) results =============
theta1_hat              = 0.404680
theta2_hat              = 0.154176
coef_on_x               = -0.404680
coef_on_rc              = 0.154176
runtime_sec_reg         = 0.0011
runtime_sec_total       = 0.0225
weighted_rmse           = 0.345944
weighted_R2_uncentered = 0.996656

First 10 OLS dependent-variable rows Y(x,g,h):
[4.5507503  4.72937526 4.87515391 4.99494402 5.09334945 5.52485739
 5.70348235 5.849261   5.96905111 6.06745654]

================ Final summary table ================
```

```
                              method  theta1_hat  theta2_hat  rho0_hat  rho1_hat
sigma_rho_hat       loglike  runtime_sec  state_space_S
                                 NFXP    0.412523    0.156883   17.8269  0.624909
4.605933 -33941.542312     0.082240             56
                     Hotz-Miller MLE    0.411447    0.156677   17.8269  0.624909
4.605933 -33941.896863     0.001535             56
            Forward simulation (c.1)    0.408617    0.155240   17.8269  0.624909
4.605933 -33943.367452     0.040132             56
            Forward simulation (c.2)    0.411057    0.158263   17.8269  0.624909
4.605933 -34007.929680     0.088821             56
Linear regression (reduced-form WLS)    0.404680    0.154176   17.8269  0.624909
4.605933           NaN     0.022480             35
```

1.7 Part (e) Policy-Inequality Estimator

We can construct the clean estimation result based on the requirement. Let the state space be $s \in \{1, \dots, S\}$, where $S = 7G$, and let the shock scale be σ_ε to denote the policy notation.

For any policy σ, define $\sigma_1(s) = \sigma(s), \quad \sigma 0(s) = 1 - \sigma(s)$. For any action $a \in \{0, 1\}$, let $\bar{u}(s; \theta)$ denotes the deterministic utility;

$$\bar{u}_0(s; \theta) = -\theta_1 x_s, \quad \bar{u}_1(s; \theta) = -\theta_2 rc_s$$

Let $\hat{F}_0, \hat{F}_1$ be the estimated transition matrices from the first step. For any policy σ, we can define the policy induced transition matrix:

$$\hat{P}_\sigma = D(\sigma_0)\hat{F}_0 + D(\sigma_1)\hat{F}_1$$

where $D(\sigma_a)$ is the diagonal matrix with entries $\sigma_a(s)$. Now define the policy specific one period expected return vector as:

$$r_\sigma(\theta) = D(\sigma_0)(\bar{u}_0(\theta) + \sigma_\varepsilon \gamma \mathbf{1} - \sigma_\varepsilon \log \sigma_0) + D(\sigma_1)(\bar{u}_1(\theta) + \sigma_\varepsilon \gamma \mathbf{1} - \sigma_\varepsilon \log \sigma_1)$$

This comes from the Hotz-Miller identity

$$\mathbb{E}[\varepsilon_a \mid a \text{ chosen }, s] = \sigma_\varepsilon \gamma - \sigma_\varepsilon \log \sigma_a(s).$$

Then the policy value vector under σ is the unique solution of

$$V_\sigma(\theta) = r_\sigma(\theta) + \beta \hat{P}_\sigma V_\sigma(\theta)$$

or equivalently,

$$V_\sigma(\theta) = (I - \beta \hat{P}_\sigma)^{-1} r_\sigma(\theta).$$

That is, once $\hat{F}_0, \hat{F}_1$ and σ are given, we can evaluate the value of that policy for any candidate θ by a linear solve, exactly in the Hotz–Miller spirit.

Now consider the observed policy as:

$$\sigma^\star(s) = \hat{P}(a = 1 \mid s)$$

where $\hat{P}$ comes from the first stage CCP estimation in part (b). Then we can construct J alternative policies, $\sigma^1, \dots, \sigma^J$. We can consider one possible alternative as $\sigma^j(s) \in [\varepsilon, 1-\varepsilon]$ for a small clipping constant $\varepsilon > 0$, so that the log terms are always well-defined.

The part (e)'s recommendations can then analogously defined as:

$$\sigma^1(s) = \min\{\sigma^\star(s) + 0.1, 1 - \varepsilon\}$$

$$\sigma^2(s) = \text{clip}(\sigma^\star(s) + \delta_{rc}(s)),$$

where $\delta_{rc}(s) > 0$ in high-rc state and $\delta_{rc}(s) < 0$ in low-rc states,

$$\sigma^3(s) = \text{clip}(\sigma^\star(s) - \delta_{rc}(s)),$$

$$\sigma^4(s) = \text{clip}(\sigma^\star(s) + \delta_x(s)),$$

where $\delta_x(s) > 0$ in high-x states and $\delta_x(s) < 0$ in low-x states,

$$\sigma^5(s) = \text{clip}(\sigma^\star(s) - \delta_x(s)),$$

Let

$$\lambda^\star(s) = \log \frac{\sigma^\star(s)}{1 - \sigma^\star(s)}.$$

Any alternative threshold perturbation $\kappa^j(s)$ defines,

$$\sigma^j(s) = \Lambda(\lambda^\star(s) + \kappa^j(s)), \quad \Lambda(u) = \frac{e^u}{1 + e^u}$$

So policy perturbations in probabilities and perturbations in threshold rules are just two equivalent ways of constructing the alternatives.

Finally, we can consider the inequality moments. For each state s and each alternative policy j, define the population inequality moment as:

$$m_j(s; \theta) = V_{\sigma^\star}(s; \theta) - V_{\sigma^j}(s; \theta).$$

The optimality of the observed policy implies

$$m_j(s;\theta_0) \geq 0, \quad \forall s, \forall j.$$

Equivalently,

$$V_{\sigma^j}(s;\theta) - V_{\sigma^\star}(s;\theta) \leq 0$$

We can consider the sample objective is:

$$\hat{Q}(\theta) = \sum_{s=1}^{S} \sum_{j=1}^{J} [\max\{0, \hat{V}_{\sigma^j}(s;\theta) - \hat{V}_{\sigma^\star}(s;\theta)\}]^2,$$

Then define the estimator by

$$\hat{\theta}^{ineq} = \arg\min_{\theta \in \Theta} \hat{Q}(\theta).$$

To sum up, the estimation procedure can be considered as:

1. Estiamte exogenous transition process for RC_t and build $\hat{F}_0, \hat{F}_1$.
2. Estimate the observed CCP $\sigma^\star(s) = \hat{P}(a = 1 \mid s)$, clipping it away from 0 and 1.
3. Construct J alternative policies $\sigma^1, \dots, \sigma^J$.
4. For any candidate θ, compute $\hat{V}_\sigma(\theta) = (I - \beta \hat{P}_\sigma)^{-1} \hat{r}_\sigma(\theta)$ for each $\sigma \in \{\sigma^\star, \sigma^1, \dots, \sigma^J\}$.
5. For each s and j, compute the violation

$$\hat{v}_j(s;\theta) = \max\{0, \hat{V}_{\sigma_j}(s;\theta) - \hat{V}_{\sigma^\star}(s;\theta)\}$$

6. Minimize

$$\hat{Q}(\theta) = \sum_{s=1}^{S} \sum_{j=1}^{J} \hat{v}_j(s;\theta)^2.$$

[6]:
```
# Part (e): Inequalities estimator based on HM steps 1-3
#
# This code reuses:
#   - F0, F1, I_S
#   - u0_base_vec, u1_base_vec
#   - p1_first_stage_vec, p0_first_stage_vec
#   - beta, gamma_euler, ccp_clip
#   - res_hm.x as the starting value
#
# The objective function is described based on the above requirement
#   min_theta sum_s sum_j [ max(0, V(s|sigma^j;theta) - V(s|sigma*;theta)) ]^2
# where sigma* is the estimated CCP and sigma^j are alternative policies.
```

```python
# 0. Controls for the inequality estimator

# sum over all states, one-state-one-vote.
# We can also use the state weight to consider the occupied states
use_state_weights = False

# Magnitude of the policy perturbations.
delta_policy = 0.10

# Optimization controls
eta_start_e = res_hm.x.copy()   # start from HM estimate
eta_bounds_e = [(-6.0, 2.0), (-6.0, 2.0)]

# Observed policy sigma* from first-stage CCPs
p1_star_vec = np.clip(p1_first_stage_vec.copy(), ccp_clip, 1.0 - ccp_clip)
p0_star_vec = 1.0 - p1_star_vec

p1_star_mat = p1_star_vec.reshape(7, K)

# Construct the 5 alternative policies recommended in the homework
#    Policy 1: add 0.10 everywhere (capped below 1)
#    Policy 2: high-RC higher, low-RC lower
#    Policy 3: opposite of policy 2
#    Policy 4: high-x higher, low-x lower
#    Policy 5: opposite of policy 4

def clip_policy(p):
    return np.clip(p, ccp_clip, 1.0 - ccp_clip)

# Centered state scores in [-1,1]
rc_score = r_grid - r_grid.mean()
rc_score = rc_score / np.max(np.abs(rc_score))
rc_dev_mat = np.tile(rc_score, (7, 1))

x_score = np.arange(1, 8, dtype=np.float64) - np.mean(np.arange(1, 8, dtype=np.
 ↪float64))
x_score = x_score / np.max(np.abs(x_score))
x_dev_mat = np.repeat(x_score[:, None], K, axis=1)

alt_policy_list = []
alt_policy_names = []

# 1. Add 0.10 to all states
alt_policy_list.append(clip_policy(p1_star_mat + delta_policy).reshape(-1))
alt_policy_names.append("Alt 1: +0.10 everywhere")

# 2. High-RC higher, low-RC lower
```

```
alt_policy_list.append(clip_policy(p1_star_mat + delta_policy * rc_dev_mat).
 ↪reshape(-1))
alt_policy_names.append("Alt 2: high-RC up, low-RC down")

# 3. Opposite of 2
alt_policy_list.append(clip_policy(p1_star_mat - delta_policy * rc_dev_mat).
 ↪reshape(-1))
alt_policy_names.append("Alt 3: high-RC down, low-RC up")

# 4. High-x higher, low-x lower
alt_policy_list.append(clip_policy(p1_star_mat + delta_policy * x_dev_mat).
 ↪reshape(-1))
alt_policy_names.append("Alt 4: high-x up, low-x down")

# 5. Opposite of 4
alt_policy_list.append(clip_policy(p1_star_mat - delta_policy * x_dev_mat).
 ↪reshape(-1))
alt_policy_names.append("Alt 5: high-x down, low-x up")

J_e = len(alt_policy_list)

# State weights in the inequality objective
#    Homework objective is unweighted over states, so default is ones.
if use_state_weights:
    # weight states by empirical visitation frequency.
    state_weight_e = N_vec / np.maximum(N_vec.sum(), 1.0)
else:
    state_weight_e = np.ones(S, dtype=np.float64)

# Policy-cache builder
#
# For each policy sigma, precompute:
#   p0, p1
#   policy-induced transition matrix:
#       P_sigma = D(p0) F0 + D(p1) F1
#   LU factorization of:
#       A_sigma = I - beta * P_sigma
#   constant derivatives:
#       dV_sigma / dtheta1 = A_sigma^{-1}[ p0 * u0_base ]
#       dV_sigma / dtheta2 = A_sigma^{-1}[ p1 * u1_base ]
#
# This makes the optimizer very fast because all policy matrices are fixed.
def build_policy_cache(p1_vec, name):
    p1_vec = clip_policy(np.asarray(p1_vec, dtype=np.float64))
    p0_vec = 1.0 - p1_vec

    neglog_p0 = -np.log(p0_vec)
```

```
    neglog_p1 = -np.log(p1_vec)

    P_sigma = p0_vec[:, None] * F0 + p1_vec[:, None] * F1
    A_sigma = I_S - beta * P_sigma
    lu_sigma, piv_sigma = lu_factor(A_sigma, check_finite=False)

    # Because A_sigma is fixed for a given policy, these are constant in theta.
    dV_dtheta1 = lu_solve(
        (lu_sigma, piv_sigma),
        p0_vec * u0_base_vec,
        check_finite=False
    )
    dV_dtheta2 = lu_solve(
        (lu_sigma, piv_sigma),
        p1_vec * u1_base_vec,
        check_finite=False
    )

    return {
        "name": name,
        "p1": p1_vec,
        "p0": p0_vec,
        "neglog_p0": neglog_p0,
        "neglog_p1": neglog_p1,
        "lu": lu_sigma,
        "piv": piv_sigma,
        "dV_dtheta1": dV_dtheta1,
        "dV_dtheta2": dV_dtheta2,
    }

# Observed policy sigma*
policy_star_cache = build_policy_cache(p1_star_vec, "Observed policy sigma*")

# Alternative policies
policy_alt_caches = [
    build_policy_cache(p1_alt, name)
    for p1_alt, name in zip(alt_policy_list, alt_policy_names)
]

# Solve value under any fixed policy cache
#
# Under the same normalization as your part-(b) code:
#   V_sigma(theta) = (I - beta P_sigma)^(-1) b_sigma(theta)
# where
#   b_sigma(theta)
#     = p0 * (theta1*u0_base + gamma - log p0)
#     + p1 * (theta2*u1_base + gamma - log p1)
```

```python
def solve_value_under_policy(theta1, theta2, cache):
    b_sigma = (
        cache["p0"] * (theta1 * u0_base_vec + gamma_euler + cache["neglog_p0"])
        + cache["p1"] * (theta2 * u1_base_vec + gamma_euler +␣
 ↪cache["neglog_p1"])
    )
    V_sigma = lu_solve((cache["lu"], cache["piv"]), b_sigma, check_finite=False)
    return V_sigma

# Inequality objective and exact gradient in eta-space
#
# Objective:
#   Q(theta) = sum_s sum_j w_s * [ max(0, V_j(s) - V_star(s)) ]^2
#
# Gradient:
#   dQ/dtheta_k
#   = 2 * sum_s sum_j w_s * 1{diff>0} * diff * d(diff)/dtheta_k
# where
#   diff = V_j - V_star
#   d(diff)/dtheta_k = dV_j/dtheta_k - dV_star/dtheta_k

def inequalities_objective_and_grad_eta(eta, cache_alt):
    theta = np.exp(eta)
    theta1 = float(theta[0])
    theta2 = float(theta[1])

    # Value under observed policy sigma*
    V_star = solve_value_under_policy(theta1, theta2, policy_star_cache)
    dVstar_dtheta1 = policy_star_cache["dV_dtheta1"]
    dVstar_dtheta2 = policy_star_cache["dV_dtheta2"]

    # Value under one alternative policy only
    V_alt = solve_value_under_policy(theta1, theta2, cache_alt)

    diff = V_alt - V_star
    active = diff > 0.0

    obj = 0.0
    grad_theta1 = 0.0
    grad_theta2 = 0.0

    if np.any(active):
        viol = diff[active]
        w = state_weight_e[active]

        dDiff_dtheta1 = cache_alt["dV_dtheta1"][active] - dVstar_dtheta1[active]
```

```
        dDiff_dtheta2 = cache_alt["dV_dtheta2"][active] - dVstar_dtheta2[active]

        obj = np.sum(w * viol**2)
        grad_theta1 = 2.0 * np.sum(w * viol * dDiff_dtheta1)
        grad_theta2 = 2.0 * np.sum(w * viol * dDiff_dtheta2)

    # Chain rule: eta = log(theta)
    grad_eta = np.array([
        grad_theta1 * theta1,
        grad_theta2 * theta2,
    ], dtype=np.float64)

    return obj, grad_eta



def run_ineq(cache_alt, eta_start):
    def single_fun(eta):
        return inequalities_objective_and_grad_eta(eta, cache_alt)

    t0 = time.perf_counter()

    res = minimize(
        single_fun,
        eta_start,
        method="L-BFGS-B",
        jac=True,
        bounds=eta_bounds_e,
        options={
            "maxiter": 200,
            "ftol": 1e-12,
            "disp": False,
        },
    )

    runtime = time.perf_counter() - t0

    theta_hat = np.exp(res.x)
    obj_val, grad_val = inequalities_objective_and_grad_eta(res.x, cache_alt)

    # Diagnostic decomposition for this one policy
    V_star_hat = solve_value_under_policy(float(theta_hat[0]),␣
↪float(theta_hat[1]), policy_star_cache)
    V_alt_hat = solve_value_under_policy(float(theta_hat[0]),␣
↪float(theta_hat[1]), cache_alt)

    diff_hat = V_alt_hat - V_star_hat
```

```
    pos_hat = np.maximum(diff_hat, 0.0)

    return {
        "method": f"Ineq single: {cache_alt['name']}",
        "policy": cache_alt["name"],
        "success": res.success,
        "message": res.message,
        "theta1_hat": float(theta_hat[0]),
        "theta2_hat": float(theta_hat[1]),
        "criterion_value": float(obj_val),
        "runtime_sec": float(runtime),
        "num_alt_policies": 1,
        "mean_violation": float(np.mean(pos_hat)),
        "max_violation": float(np.max(pos_hat)),
        "num_violating_states": int(np.sum(pos_hat > 0.0)),
        "eta_hat": res.x.copy(),
    }

# Run all five single-policy estimators
#    Warm-start each run from HM to keep interpretation clean.
single_results = []

for cache_alt in policy_alt_caches:
    out = run_ineq(cache_alt, eta_start=res_hm.x.copy())
    single_results.append(out)

single_report = pd.DataFrame([{
    "method": r["method"],
    "theta1_hat": r["theta1_hat"],
    "theta2_hat": r["theta2_hat"],
    "criterion_value": r["criterion_value"],
    "runtime_sec": r["runtime_sec"],
    "num_alt_policies": r["num_alt_policies"],
    "mean_violation": r["mean_violation"],
    "max_violation": r["max_violation"],
    "num_violating_states": r["num_violating_states"],
    "success": r["success"],
} for r in single_results])

print("\n================ Part (e) single-policy-only results ================")
print(single_report.to_string(index=False))

# A compact diagnostic table focusing only on policy comparison
single_policy_diagnostics = pd.DataFrame([{
    "policy": r["policy"],
    "theta1_hat": r["theta1_hat"],
    "theta2_hat": r["theta2_hat"],
```

```
    "criterion_value": r["criterion_value"],
    "mean_violation": r["mean_violation"],
    "max_violation": r["max_violation"],
    "num_violating_states": r["num_violating_states"],
    "runtime_sec": r["runtime_sec"],
} for r in single_results])

print("\n================ Part (e) single-policy diagnostics ================")
print(single_policy_diagnostics.to_string(index=False))

# Append all five single-policy estimates to summary_all
summary_part_e_single = pd.DataFrame({
    "method": [r["method"] for r in single_results],
    "theta1_hat": [r["theta1_hat"] for r in single_results],
    "theta2_hat": [r["theta2_hat"] for r in single_results],
    "rho0_hat": [rho0_hat] * len(single_results),
    "rho1_hat": [rho1_hat] * len(single_results),
    "sigma_rho_hat": [sigma_rho_hat] * len(single_results),
    "loglike": [np.nan] * len(single_results),   # part (e) is not MLE
    "runtime_sec": [r["runtime_sec"] for r in single_results],
    "state_space_S": [S] * len(single_results),
    "criterion_value": [r["criterion_value"] for r in single_results],
    "num_alt_policies": [1] * len(single_results),
})

if "summary_all" in globals():
    for col in ["criterion_value", "num_alt_policies"]:
        if col not in summary_all.columns:
            summary_all[col] = np.nan
    summary_all = pd.concat([summary_all, summary_part_e_single],␣
 ↪ignore_index=True)
elif "summary" in globals():
    summary_tmp = summary.copy()
    for col in ["criterion_value", "num_alt_policies"]:
        if col not in summary_tmp.columns:
            summary_tmp[col] = np.nan
    summary_all = pd.concat([summary_tmp, summary_part_e_single],␣
 ↪ignore_index=True)
else:
    summary_all = summary_part_e_single.copy()

print("\n================ Final summary table with single-policy part (e)␣
 ↪================")
print(summary_all.to_string(index=False))
```

```
================ Part (e) single-policy-only results ================
```

```
                                    method  theta1_hat  theta2_hat
criterion_value  runtime_sec  num_alt_policies  mean_violation  max_violation
num_violating_states  success
       Ineq single: Alt 1: +0.10 everywhere    0.411447    0.156677
0.0     0.000400                 1             0.0            0.0
0     True
Ineq single: Alt 2: high-RC up, low-RC down    0.411447    0.156677
0.0     0.000318                 1             0.0            0.0
0     True
Ineq single: Alt 3: high-RC down, low-RC up    0.411447    0.156677
0.0     0.000327                 1             0.0            0.0
0     True
  Ineq single: Alt 4: high-x up, low-x down    0.411447    0.156677
0.0     0.000232                 1             0.0            0.0
0     True
  Ineq single: Alt 5: high-x down, low-x up    0.411447    0.156677
0.0     0.000195                 1             0.0            0.0
0     True

================ Part (e) single-policy diagnostics ================
                        policy  theta1_hat  theta2_hat  criterion_value
mean_violation  max_violation  num_violating_states  runtime_sec
       Alt 1: +0.10 everywhere    0.411447    0.156677              0.0
0.0            0.0                     0     0.000400
Alt 2: high-RC up, low-RC down    0.411447    0.156677              0.0
0.0            0.0                     0     0.000318
Alt 3: high-RC down, low-RC up    0.411447    0.156677              0.0
0.0            0.0                     0     0.000327
  Alt 4: high-x up, low-x down    0.411447    0.156677              0.0
0.0            0.0                     0     0.000232
  Alt 5: high-x down, low-x up    0.411447    0.156677              0.0
0.0            0.0                     0     0.000195

================ Final summary table with single-policy part (e)
================
                                    method  theta1_hat  theta2_hat  rho0_hat
rho1_hat  sigma_rho_hat       loglike  runtime_sec  state_space_S
criterion_value  num_alt_policies
                                      NFXP    0.412523    0.156883   17.8269
0.624909       4.605933 -33941.542312     0.082240             56
NaN               NaN
                           Hotz-Miller MLE    0.411447    0.156677   17.8269
0.624909       4.605933 -33941.896863     0.001535             56
NaN               NaN
                    Forward simulation (c.1)    0.408617    0.155240   17.8269
0.624909       4.605933 -33943.367452     0.040132             56
NaN               NaN
                    Forward simulation (c.2)    0.411057    0.158263   17.8269
```

```
0.624909       4.605933 -34007.929680     0.088821             56
NaN               NaN
       Linear regression (reduced-form WLS)    0.404680    0.154176   17.8269
0.624909       4.605933           NaN     0.022480             35
NaN               NaN
       Ineq single: Alt 1: +0.10 everywhere    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000400             56
0.0               1.0
Ineq single: Alt 2: high-RC up, low-RC down    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000318             56
0.0               1.0
Ineq single: Alt 3: high-RC down, low-RC up    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000327             56
0.0               1.0
  Ineq single: Alt 4: high-x up, low-x down    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000232             56
0.0               1.0
  Ineq single: Alt 5: high-x down, low-x up    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000195             56
0.0               1.0
```

1.8 Additional Consideration using Forward Simulation

In terms of the using the estimation method like the part (c. 1) and (c. 2) the results are also analogously defined since the methods only leverage the forward simulation method and we also don't need to replace the objective function with the above.

Fix any policy σ, any state s, and any candidate parameter vector θ. Let

$$s_0^{(r)} = s, \quad r = 1, \dots, R$$

and simulate a path $\{s_\tau^{(r)}, a_\tau^{(r)}\}_{\tau=0}^{T_{\text{sim}}}$ under the policy σ. At each simulated date τ, draw the action directly from the policy,

$$a_\tau^{(r)} \sim \text{Bernoulli}(\sigma_1(s_\tau^{(r)})).$$

Then update the state using the estimated transition law $\hat{F}_a$, equivalently the deterministic mileage transitioan and the estimated reduced form process for RC.

with the shock scale σ_ε, we can write

$$G_\sigma^{(1,r)}(s;\theta) = \sum_{\tau=0}^{T_{sim}} \beta^\tau [\bar{u}(s_\tau^{(r)}, a_\tau^{(r)}; \theta) + \sigma_\varepsilon \gamma - \sigma_\varepsilon \log \sigma_{a_\tau^{(r)}} s_\tau^{(r)})].$$

Then define the simulation based policy value estimator as:

$$\hat{V}_{\sigma}^{(1)}(s;\theta) = \frac{1}{R}\sum_{r=1}^{R} G_{\sigma}^{(1,r)}(s;\theta)$$

As $R \to \infty$ and $T_{sim} \to \infty$ this converges to the policy value $V_{\sigma}(s;\theta)$, under the same regulatory conditions in the part (c. 1)

The Part (c. 2) way of extension can also be constructed, at each simulated date τ, draw $\varepsilon_{0\tau}^{(r)}, \varepsilon_{1\tau}^{(r)}$ independently from the Type-I extreme value distribution. The replacement effect is:

$$a_{\tau}^{(r)} = 1 \iff \log\frac{\sigma_1(s_{\tau}^{(r)})}{\sigma_0(s_{\tau}^{(r)})} + \varepsilon_{1\tau}^{(r)} - \varepsilon_{0\tau}^{(r)} > 0.$$

Then the simulated truncated return can be found as:

$$G_{\sigma}^{(2,r)}(s;\theta) = \sum_{\tau=0}^{T_{sim}} \beta^{\tau}[\bar{u}(s_{\tau}^{(r)}, a_{\tau}^{(r)};\theta) + \varepsilon_{a_{\tau}^{(r)},\tau}^{(r)}].$$

and the corresponding simulation based estimator is the policy value as:

$$\hat{V}_{\sigma}^{(1)}(s;\theta) = \frac{1}{R}\sum_{r=1}^{R} G_{\sigma}^{(2,r)}(s;\theta)$$

Once we have the results, we can correspondingly define

$$\hat{Q}^{(1)}(\theta) = \sum_{s=1}^{S}\sum j = 1^{J}[\max\{0, \hat{V}_{\sigma^j}^{(1)}(s;\theta) - \hat{V}_{\sigma^{\star}}^{(1)}(s;\theta)\}]^2$$

and estimate

$$\hat{\theta}_{ineq}^{(1)} = \arg\min_{\theta\in\Theta} \hat{Q}^{(1)}(\theta).$$

and similarly for the case of $\hat{\theta}_{ineq}^{(2)}$ which can be defined analogously.

The baseline simulation based inequality estimator should sum over all possible states; but summing over the observed state evolution path is an alternative weighted version, and it may not satisfy the global optimality criterion.

1.9 Final Discussion

NFXP and HM MLE are essentially giving the same structural estimate, and the (C. 2) forward simulation gives a noiser estimation.

In terms of the tradeoff:

For matrix size and memory, NFXP is the heaviest conceptually because it works with the full state space and repeatedly applies the Bellman operator.

In terms of CCP reliance, NFXP relies the least on a first-stage CCP estimate. That is its main strength. It uses the structural Bellman fixed point directly rather than taking estimated CCPs as an input. So if some states are rarely visited, NFXP is less vulnerable than HM, forward simulation, or regression to noisy CCP estimates. It still depends on the estimated RC transition process and on state aggregation, but not on a first-stage CCP smoother.

In terms of time, it is slower than HM but still fast in the current small state space: about 0.236 seconds. That is because it solves the inner fixed point repeatedly. As the state space grows, NFXP is usually the first method to become computationally burdensome.

HM involves in the matrix inversion or linear solves, so when the state space becomes very large, cubic scaling of dense matrix methods becomes the main concern.

For CCP reliance, HM depends heavily on a good CCP estimate. This is the central tradeoff. The value is recovered by plugging estimated CCPs into the inversion formula. If some states are rarely visited, the estimated CCP can be noisy or even hit 0 or 1, which then contaminates the inversion because terms like $-\log P(a \mid s)$. Then smoothing should be considered.

For matrix size, (c.1) is attractive when the state space becomes large because it does not require solving the Bellman fixed point and can avoid large matrix inversion if implemented more generally by simulation.

FOr CCP reliance, (c. 1) depends strongly on the estimated CCP because it draws actions from the estimated policy $\widehat{P}(a \mid s)$. So if some states are rare and CCP estimate is poor, the simulated value is poor. It's worth mentioning that the cost of running time is high because the running time is like $S \times N_{sim} \times T_{sim}$, so it can become expensive even when matrix inversion is avoided.

(c. 2) is also similar to the (c. 1) and it's worth mentioning that it is usually less attractive unless there is a specific reason to keep the original random-utility representation explicit.

In terms of the OLS. For CCP reliance, this method depends very heavily on CCP quality, arguably more than HM in practice. The regression uses smoothed CCPs as direct inputs in log-odds and continuation-proxy terms. If some states are rare, then the CCP estimates can be unstable, and the logarithms amplify that instability. The solution notes explicitly discuss this small-sample problem and recommend smoothing the CCP using parametric or spline-based methods. Of course, for the matrix size, this is the fastest and since this is just an OLS. -_-

Single policy inequality methods use the similar idea from the HM inversion. The difference is that it compares values across policies instead of maximizing a likelihood. For CCP reliance, part (e) depends extremely heavily on CCP quality because the observed policy $\sigma^\star$ is itself the first stage CCP-estimate, and every perturbation policy is constructed from it. So in small samples or sparse states, part (e) is just as sensitive as HM, possibly even more so, because the observed CCP is the central object being declared optimal.

1.10 Apendix A

Deriving the distribution of the maximum:

Denote $M = \max_a(\delta_a + \varepsilon_a)$ where in the case of our specific question $\delta_a = \bar{u}(s,a) + \beta EV_a(s)$. Then we have:

$$\Pr(M \leq m) = \Pr(\delta_a + \varepsilon_a \leq m \ \forall a)$$

Equivalently,

$$\Pr(M \leq m) = \prod_a \Pr(\varepsilon_a \leq m - \delta_a)$$

using independence condition. Then for type-1 extreme distribution with cdf, we have:

$$F(\varepsilon) = \exp(-e^{-\varepsilon})$$

Plugging back, we have:

$$\Pr(M \leq m) = \exp(-e^{-m} \sum_a e^{\delta_a}) = \exp(-\exp[-(m - \log \sum_a \delta^a)])$$

So the maximum M is itself Gumbel-distributed with location $\mu = \log \sum_a e^{\delta_a}$. Therefore, we can write:

$$\mathbb{E}[M] = \log \sum_a e^{\delta_a} + \gamma$$

Substitute back with δ_a then we can derive the result.

1.11 Appendix B

system information:

```
=== OS AND KERNEL ===
PRETTY_NAME="Zorin OS 18"
6.17.0-19-generic

=== CPU TOPOLOGY & CACHES ===
CPU(s):                                  8
Model name:                              11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Thread(s) per core:                      2
Core(s) per socket:                      4
Socket(s):                               1
L1d cache:                               192 KiB (4 instances)
L1i cache:                               128 KiB (4 instances)
L2 cache:                                5 MiB (4 instances)
L3 cache:                                8 MiB (1 instance)
NUMA node(s):                            1
NUMA node0 CPU(s):                       0-7

=== MEMORY (RAM) ===
               total        used        free      shared  buff/cache   available
Mem:            15Gi       6.8Gi       1.1Gi       659Mi       8.6Gi       8.5Gi
Swap:          2.0Gi       852Ki       2.0Gi
```

1.12 Appendix C

This is the sample based estimation in the part (c. 1) and (c. 2).

The two implementations that in the main writeup and the appendix use the same part-(c) estimator in spirit — both precompute the θ-invariant simulation objects once and then optimize without resimulating — but they differ in the **first-stage objects and therefore in the economic approximation they are using**. The implementation above keeps the same first-stage structure as parts (a) and (b): it uses the AR(1)-implied discretized RC transition matrix Q_{rc}, the smoothed CCP $\hat{P}(a \mid s)$ on the (x, rc) state grid, and the same structural state-transition matrices F_0, F_1, so it is internally consistent with the rest of the code and closer to a single coherent estimation framework. (This is the reason I list it in the main text, since the later code also inherent the same structure). By contrast, the version below changes the first stage to use the empirical RC-bin transition matrix computed directly from observed bin-to-bin moves and the **raw clipped frequency CCP** on the $(x, \text{RC-bin})$ grid, then applies the same precompute-once simulation trick. So the main difference is not the simulation shortcut, but that this version is a **sample-replication implementation**.

[7]:
```
#   - empirical RC-bin transition matrix Pi
#   - raw clipped frequency CCP on the (x, rc_bin) grid
#   - one-time simulation precomputation outside the optimizer
#   - Nelder-Mead optimization
#
# This is the right block if your goal is to reproduce the
# Qc1_pv and Qc2_pv results as closely as possible.

# ------------------------------------------------------------
# 0. Global constants for the sample-style part (c)
# ------------------------------------------------------------
BETA_pv = beta
N_X_pv = 7
N_RC_BINS_pv = K           # keep K = 8 to match your current setup and the␣
 ↪sample notebook
S_pv = N_X_pv * N_RC_BINS_pv
GAMMA_pv = gamma_euler

# Match notebook tuning
N_SIM_c1_pv = 100
T_SIM_c1_pv = 120
SIM_SEED_c1_pv = 12345

N_SIM_c2_pv = 100
T_SIM_c2_pv = 120
SIM_SEED_c2_pv = 24680

theta_init_pv = theta0.copy()

# ------------------------------------------------------------
```

```
# 1. Build the empirical RC bins and empirical RC transition matrix Pi
#    This matches the notebook implementation, not your AR(1)-based Q_rc.
# ------------------------------------------------------------
rc_min_pv = float(df["rc"].min())
rc_max_pv = float(df["rc"].max())

bin_edges_pv = np.linspace(rc_min_pv, rc_max_pv, N_RC_BINS_pv + 1)
rc_grid_pv = 0.5 * (bin_edges_pv[:-1] + bin_edges_pv[1:])

# Assign each row to an RC bin (0-indexed), same logic as the notebook.
rc_bin_idx_pv = np.searchsorted(bin_edges_pv[1:], df["rc"].to_numpy(dtype=np.
 ↪float64), side="left")
rc_bin_idx_pv = np.clip(rc_bin_idx_pv, 0, N_RC_BINS_pv - 1).astype(np.int16)

df_pv = df.copy()
df_pv["rc_bin_pv"] = rc_bin_idx_pv
df_pv["rc_mid_pv"] = rc_grid_pv[rc_bin_idx_pv]

# Empirical RC transition matrix Pi from observed bin-to-bin transitions.
C_pv = np.zeros((N_RC_BINS_pv, N_RC_BINS_pv), dtype=np.float64)
for _, g in df_pv.groupby("i"):
    bins = g["rc_bin_pv"].to_numpy(dtype=np.int16)
    for j_from, j_to in zip(bins[:-1], bins[1:]):
        C_pv[j_from, j_to] += 1.0

Pi_pv = C_pv / C_pv.sum(axis=1, keepdims=True)

# Continuous AR(1) estimates are still reported for comparability.
# Reuse your already-estimated rho0_hat, rho1_hat, sigma_rho_hat.

# ------------------------------------------------------------
# 2. Build full state space and action-specific transitions TP0 / TP1
#    This uses empirical Pi rather than your F0/F1 from Q_rc.
# ------------------------------------------------------------
xi_of_s_pv = np.array([xi for xi in range(N_X_pv) for _ in␣
 ↪range(N_RC_BINS_pv)], dtype=np.int16)
j_of_s_pv  = np.array([j for _ in range(N_X_pv) for j in range(N_RC_BINS_pv)],␣
 ↪dtype=np.int16)

x_of_s_pv = xi_of_s_pv + 1
rc_of_s_pv = rc_grid_pv[j_of_s_pv]

xi_next_mnt_pv = np.minimum(xi_of_s_pv + 1, N_X_pv - 1)
xi_next_rep_pv = np.zeros(S_pv, dtype=np.int16)

TP0_pv = np.zeros((S_pv, S_pv), dtype=np.float64)
TP1_pv = np.zeros((S_pv, S_pv), dtype=np.float64)
```

```
for s in range(S_pv):
    j = j_of_s_pv[s]

    col0 = int(xi_next_mnt_pv[s]) * N_RC_BINS_pv
    TP0_pv[s, col0:col0 + N_RC_BINS_pv] = Pi_pv[j, :]

    col1 = int(xi_next_rep_pv[s]) * N_RC_BINS_pv
    TP1_pv[s, col1:col1 + N_RC_BINS_pv] = Pi_pv[j, :]

# ------------------------------------------------------------
# 3. State-level counts and raw clipped frequency CCP
#    This matches the notebook implementation, not your smoothed CCP.
# ------------------------------------------------------------
N1_pv = np.zeros((N_X_pv, N_RC_BINS_pv), dtype=np.float64)
N0_pv = np.zeros((N_X_pv, N_RC_BINS_pv), dtype=np.float64)

for xi_obs, j_obs, a_obs in zip(
    df_pv["x"].to_numpy(dtype=np.int16) - 1,
    df_pv["rc_bin_pv"].to_numpy(dtype=np.int16),
    df_pv["a"].to_numpy(dtype=np.int8),
):
    if a_obs == 1:
        N1_pv[xi_obs, j_obs] += 1.0
    else:
        N0_pv[xi_obs, j_obs] += 1.0

N_total_pv = N0_pv + N1_pv

eps_ccp_pv = 1e-6
P1_hat_pv = np.where(N_total_pv > 0, N1_pv / N_total_pv, eps_ccp_pv)
P1_hat_pv = np.clip(P1_hat_pv, eps_ccp_pv, 1.0 - eps_ccp_pv)
P0_hat_pv = 1.0 - P1_hat_pv

P1_flat_pv = P1_hat_pv.ravel()
P0_flat_pv = P0_hat_pv.ravel()
log_odds_flat_pv = np.log(P1_flat_pv) - np.log(P0_flat_pv)

occupied_pv = (N_total_pv.ravel() > 0)

# ------------------------------------------------------------
# 4. Common objects for simulation
# ------------------------------------------------------------
discounts_c1_pv = BETA_pv ** np.arange(T_SIM_c1_pv)
discounts_c2_pv = BETA_pv ** np.arange(T_SIM_c2_pv)

cumTP0_pv = np.cumsum(TP0_pv, axis=1)
```

```python
cumTP1_pv = np.cumsum(TP1_pv, axis=1)
cumTP0_pv[:, -1] = 1.0
cumTP1_pv[:, -1] = 1.0

# ------------------------------------------------------------
# 5. Part (c.1): notebook-style precomputation
#    V_hat(s;theta) = -theta1 * maint_component
#                     -theta2 * repl_component
#                     +corr_component
# ------------------------------------------------------------
def simulate_value_components_c1_pv(
    P1_flat,
    P0_flat,
    x_of_s,
    rc_of_s,
    cumTP0,
    cumTP1,
    n_sim=N_SIM_c1_pv,
    t_sim=T_SIM_c1_pv,
    seed=SIM_SEED_c1_pv,
):
    rng = np.random.default_rng(seed)

    maint_component = np.zeros(S_pv, dtype=np.float64)
    repl_component  = np.zeros(S_pv, dtype=np.float64)
    corr_component  = np.zeros(S_pv, dtype=np.float64)

    for s0 in range(S_pv):
        states = np.full(n_sim, s0, dtype=np.int32)

        maint_acc = np.zeros(n_sim, dtype=np.float64)
        repl_acc  = np.zeros(n_sim, dtype=np.float64)
        corr_acc  = np.zeros(n_sim, dtype=np.float64)

        for tau in range(t_sim):
            beta_tau = discounts_c1_pv[tau]

            p1 = P1_flat[states]
            p0 = P0_flat[states]

            u_action = rng.random(n_sim)
            a1 = u_action < p1
            a0 = ~a1

            p_selected = np.where(a1, p1, p0)

            maint_acc += beta_tau * x_of_s[states] * a0
```

```
            repl_acc  += beta_tau * rc_of_s[states] * a1
            corr_acc  += beta_tau * (GAMMA_pv - np.log(p_selected))

            u_next = rng.random(n_sim)
            cdf = np.where(a1[:, None], cumTP1[states, :], cumTP0[states, :])
            next_states = (u_next[:, None] > cdf).sum(axis=1)
            next_states = np.minimum(next_states, S_pv - 1)
            states = next_states

        maint_component[s0] = maint_acc.mean()
        repl_component[s0]  = repl_acc.mean()
        corr_component[s0]  = corr_acc.mean()

    return maint_component, repl_component, corr_component

t0 = time.perf_counter()
maint_component_c1_pv, repl_component_c1_pv, corr_component_c1_pv =␣
 ↪simulate_value_components_c1_pv(
    P1_flat=P1_flat_pv,
    P0_flat=P0_flat_pv,
    x_of_s=x_of_s_pv,
    rc_of_s=rc_of_s_pv,
    cumTP0=cumTP0_pv,
    cumTP1=cumTP1_pv,
    n_sim=N_SIM_c1_pv,
    t_sim=T_SIM_c1_pv,
    seed=SIM_SEED_c1_pv,
)
time_precompute_c1_pv = time.perf_counter() - t0

def simulated_value_c1_pv(theta1, theta2):
    return -theta1 * maint_component_c1_pv - theta2 * repl_component_c1_pv +␣
 ↪corr_component_c1_pv

def model_ccp_c1_pv(theta1, theta2):
    V_hat = simulated_value_c1_pv(theta1, theta2)

    EV0 = TP0_pv @ V_hat
    EV1 = TP1_pv @ V_hat

    delta0 = -theta1 * x_of_s_pv + BETA_pv * EV0
    delta1 = -theta2 * rc_of_s_pv + BETA_pv * EV1

    dmax = np.maximum(delta0, delta1)
    exp0 = np.exp(delta0 - dmax)
    exp1 = np.exp(delta1 - dmax)
    P1_model = exp1 / (exp0 + exp1)
```

```
    return V_hat, P1_model.reshape(N_X_pv, N_RC_BINS_pv)

def neg_log_likelihood_c1_pv(params):
    theta1, theta2 = params
    if theta1 <= 0 or theta2 <= 0:
        return 1e10

    _, P1_model = model_ccp_c1_pv(theta1, theta2)
    P0_model = 1.0 - P1_model

    eps = 1e-12
    ll = np.sum(N1_pv * np.log(P1_model + eps) + N0_pv * np.log(P0_model + eps))
    return -ll

print("\n--- Part (c.1) sample-style forward simulation ---")
print(f"Initial theta = {theta_init_pv}")
print(f"Initial NLL   = {neg_log_likelihood_c1_pv(theta_init_pv):.6f}")

t0 = time.perf_counter()
result_c1_pv = minimize(
    neg_log_likelihood_c1_pv,
    theta_init_pv,
    method="Nelder-Mead",
    options=dict(maxiter=5000, xatol=1e-8, fatol=1e-8, disp=False),
)
time_opt_c1_pv = time.perf_counter() - t0
time_total_c1_pv = time_precompute_c1_pv + time_opt_c1_pv

theta1_c1_pv, theta2_c1_pv = result_c1_pv.x
V_hat_c1_pv, P1_final_c1_pv = model_ccp_c1_pv(theta1_c1_pv, theta2_c1_pv)
ll_c1_pv = -result_c1_pv.fun

print("\n============= Sample-style forward simulation (c.1) results␣
↪=============")
print(f"theta1_hat             = {theta1_c1_pv:.6f}")
print(f"theta2_hat             = {theta2_c1_pv:.6f}")
print(f"loglike                = {ll_c1_pv:.6f}")
print(f"converged              = {result_c1_pv.success}")
print(f"optimizer_iterations   = {result_c1_pv.nit}")
print(f"time_precompute_sec    = {time_precompute_c1_pv:.4f}")
print(f"time_optimize_sec      = {time_opt_c1_pv:.4f}")
print(f"runtime_sec_total      = {time_total_c1_pv:.4f}")

# ------------------------------------------------------------
# 6. Part (c.2): notebook-style precomputation
#    V_hat(s;theta) = -theta1 * maint_component
```

```
#                       -theta2 * repl_component
#                       +shock_component
# ---------------------------------------------------------
def simulate_value_components_c2_pv():
    rng = np.random.default_rng(SIM_SEED_c2_pv)

    maint_component = np.zeros(S_pv, dtype=np.float64)
    repl_component  = np.zeros(S_pv, dtype=np.float64)
    shock_component = np.zeros(S_pv, dtype=np.float64)

    for s0 in range(S_pv):
        states = np.full(N_SIM_c2_pv, s0, dtype=np.int32)

        maint_acc = np.zeros(N_SIM_c2_pv, dtype=np.float64)
        repl_acc  = np.zeros(N_SIM_c2_pv, dtype=np.float64)
        shock_acc = np.zeros(N_SIM_c2_pv, dtype=np.float64)

        for tau in range(T_SIM_c2_pv):
            beta_tau = discounts_c2_pv[tau]

            eps0 = rng.gumbel(loc=0.0, scale=1.0, size=N_SIM_c2_pv)
            eps1 = rng.gumbel(loc=0.0, scale=1.0, size=N_SIM_c2_pv)

            log_odds = log_odds_flat_pv[states]
            a1 = (log_odds + eps1 - eps0) > 0.0
            a0 = ~a1

            maint_acc += beta_tau * x_of_s_pv[states] * a0
            repl_acc  += beta_tau * rc_of_s_pv[states] * a1
            shock_acc += beta_tau * np.where(a1, eps1, eps0)

            u_next = rng.random(N_SIM_c2_pv)
            cdf = np.where(a1[:, None], cumTP1_pv[states, :], cumTP0_pv[states,␣
 ↪:])
            next_states = (u_next[:, None] > cdf).sum(axis=1)
            next_states = np.minimum(next_states, S_pv - 1)
            states = next_states

        maint_component[s0] = maint_acc.mean()
        repl_component[s0]  = repl_acc.mean()
        shock_component[s0] = shock_acc.mean()

    return maint_component, repl_component, shock_component

t0 = time.perf_counter()
maint_component_c2_pv, repl_component_c2_pv, shock_component_c2_pv =␣
 ↪simulate_value_components_c2_pv()
```

```python
time_precompute_c2_pv = time.perf_counter() - t0

def simulated_value_c2_pv(theta1, theta2):
    return -theta1 * maint_component_c2_pv - theta2 * repl_component_c2_pv +␣
 ↪shock_component_c2_pv

def model_ccp_c2_pv(theta1, theta2):
    V_hat = simulated_value_c2_pv(theta1, theta2)

    EV0 = TP0_pv @ V_hat
    EV1 = TP1_pv @ V_hat

    delta0 = -theta1 * x_of_s_pv + BETA_pv * EV0
    delta1 = -theta2 * rc_of_s_pv + BETA_pv * EV1

    dmax = np.maximum(delta0, delta1)
    exp0 = np.exp(delta0 - dmax)
    exp1 = np.exp(delta1 - dmax)
    P1_model = exp1 / (exp0 + exp1)

    return V_hat, P1_model.reshape(N_X_pv, N_RC_BINS_pv)

def neg_log_likelihood_c2_pv(params):
    theta1, theta2 = params
    if theta1 <= 0 or theta2 <= 0:
        return 1e10

    _, P1_model = model_ccp_c2_pv(theta1, theta2)
    P0_model = 1.0 - P1_model

    eps = 1e-12
    ll = np.sum(N1_pv * np.log(P1_model + eps) + N0_pv * np.log(P0_model + eps))
    return -ll

print("\n--- Part (c.2) sample-style forward simulation ---")
print(f"Initial theta = {theta_init_pv}")
print(f"Initial NLL   = {neg_log_likelihood_c2_pv(theta_init_pv):.6f}")

t0 = time.perf_counter()
result_c2_pv = minimize(
    neg_log_likelihood_c2_pv,
    theta_init_pv,
    method="Nelder-Mead",
    options=dict(maxiter=5000, xatol=1e-8, fatol=1e-8, disp=False),
)
time_opt_c2_pv = time.perf_counter() - t0
time_total_c2_pv = time_precompute_c2_pv + time_opt_c2_pv
```

```
theta1_c2_pv, theta2_c2_pv = result_c2_pv.x
V_hat_c2_pv, P1_final_c2_pv = model_ccp_c2_pv(theta1_c2_pv, theta2_c2_pv)
ll_c2_pv = -result_c2_pv.fun

print("\n============= Sample-style forward simulation (c.2) results␣
↪=============")
print(f"theta1_hat             = {theta1_c2_pv:.6f}")
print(f"theta2_hat             = {theta2_c2_pv:.6f}")
print(f"loglike                = {ll_c2_pv:.6f}")
print(f"converged              = {result_c2_pv.success}")
print(f"optimizer_iterations   = {result_c2_pv.nit}")
print(f"time_precompute_sec    = {time_precompute_c2_pv:.4f}")
print(f"time_optimize_sec      = {time_opt_c2_pv:.4f}")
print(f"runtime_sec_total      = {time_total_c2_pv:.4f}")

# ------------------------------------------------------------
# 7. Append notebook-style part (c) results to your summary table
# ------------------------------------------------------------
summary_part_c_pv = pd.DataFrame({
    "method": ["Forward simulation (c.1) sample-style", "Forward simulation (c.
↪2) sample-style"],
    "theta1_hat": [theta1_c1_pv, theta1_c2_pv],
    "theta2_hat": [theta2_c1_pv, theta2_c2_pv],
    "rho0_hat": [rho0_hat, rho0_hat],
    "rho1_hat": [rho1_hat, rho1_hat],
    "sigma_rho_hat": [sigma_rho_hat, sigma_rho_hat],
    "loglike": [ll_c1_pv, ll_c2_pv],
    "runtime_sec": [time_total_c1_pv, time_total_c2_pv],
    "state_space_S": [S_pv, S_pv],
})

if "summary_all" in globals():
    # avoid duplicate rows if you rerun the cell
    summary_all = pd.concat([summary_all, summary_part_c_pv], ignore_index=True)
elif "summary" in globals():
    summary_all = pd.concat([summary, summary_part_c_pv], ignore_index=True)
else:
    summary_all = summary_part_c_pv.copy()

print("\n================ Summary table with sample-style part (c)␣
↪================")
print(summary_all.to_string(index=False))

print("\nFirst 10 occupied-state notebook-style CCPs:")
print(P1_flat_pv[occupied_pv][:10])
```

```python
print("\nFirst 10 occupied-state model CCPs from sample-style (c.1):")
print(P1_final_c1_pv.ravel()[occupied_pv][:10])

print("\nFirst 10 occupied-state model CCPs from sample-style (c.2):")
print(P1_final_c2_pv.ravel()[occupied_pv][:10])
```

```
--- Part (c.1) sample-style forward simulation ---
Initial theta = [0.3  0.35]
Initial NLL   = 86406.060751

============= Sample-style forward simulation (c.1) results =============
theta1_hat             = 0.395534
theta2_hat             = 0.153706
loglike                = -34001.052261
converged              = True
optimizer_iterations   = 66
time_precompute_sec    = 0.2837
time_optimize_sec      = 0.0055
runtime_sec_total      = 0.2892

--- Part (c.2) sample-style forward simulation ---
Initial theta = [0.3  0.35]
Initial NLL   = 82597.238441

============= Sample-style forward simulation (c.2) results =============
theta1_hat             = 0.389652
theta2_hat             = 0.153694
loglike                = -34408.221935
converged              = True
optimizer_iterations   = 67
time_precompute_sec    = 0.3152
time_optimize_sec      = 0.0053
runtime_sec_total      = 0.3205

================ Summary table with sample-style part (c) ================
                                     method  theta1_hat  theta2_hat  rho0_hat
rho1_hat  sigma_rho_hat       loglike  runtime_sec  state_space_S
criterion_value  num_alt_policies
                                       NFXP    0.412523    0.156883   17.8269
0.624909       4.605933 -33941.542312     0.082240             56
NaN               NaN
                           Hotz-Miller MLE    0.411447    0.156677   17.8269
0.624909       4.605933 -33941.896863     0.001535             56
NaN               NaN
                    Forward simulation (c.1)    0.408617    0.155240   17.8269
0.624909       4.605933 -33943.367452     0.040132             56
NaN               NaN
```

```
                    Forward simulation (c.2)    0.411057    0.158263   17.8269
0.624909       4.605933 -34007.929680     0.088821             56
NaN               NaN
       Linear regression (reduced-form WLS)    0.404680    0.154176   17.8269
0.624909       4.605933           NaN     0.022480             35
NaN               NaN
       Ineq single: Alt 1: +0.10 everywhere    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000400             56
0.0               1.0
Ineq single: Alt 2: high-RC up, low-RC down    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000318             56
0.0               1.0
Ineq single: Alt 3: high-RC down, low-RC up    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000327             56
0.0               1.0
  Ineq single: Alt 4: high-x up, low-x down    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000232             56
0.0               1.0
  Ineq single: Alt 5: high-x down, low-x up    0.411447    0.156677   17.8269
0.624909       4.605933           NaN     0.000195             56
0.0               1.0
      Forward simulation (c.1) sample-style    0.395534    0.153706   17.8269
0.624909       4.605933 -34001.052261     0.289190             56
NaN               NaN
      Forward simulation (c.2) sample-style    0.389652    0.153694   17.8269
0.624909       4.605933 -34408.221935     0.320497             56
NaN               NaN

First 10 occupied-state notebook-style CCPs:
[2.84900285e-02 1.93765796e-02 9.29487179e-03 5.36978277e-03
 3.65230095e-03 2.49376559e-03 1.00000000e-06 1.00000000e-06
 7.76340111e-02 6.64023786e-02]

First 10 occupied-state model CCPs from sample-style (c.1):
[0.0280787  0.01671116 0.00941683 0.00586964 0.00326632 0.00189347
 0.00102983 0.00061493 0.09076108 0.05708764]

First 10 occupied-state model CCPs from sample-style (c.2):
[0.02036461 0.00970453 0.01039406 0.0059011  0.00314511 0.0016803
 0.00088956 0.00037213 0.10873685 0.05377112]
```

1.13 Appendix D

The Tauchen's implementation

Start from

$$RC_t = \rho_0 + \rho_1 RC_{t-1} + e_t, \qquad e_t \sim N(0, \sigma_\rho^2),$$

with $|\rho_1| < 1$. The stationary mean and stationary standard deviation are

$$\mu_{RC} = \frac{\rho_0}{1-\rho_1}, \qquad \sigma_{RC,\infty} = \frac{\sigma_\rho}{\sqrt{1-\rho_1^2}}.$$

Tauchen chooses G grid points for RC centered around the stationary mean:

$$r^1, \dots, r^G, \qquad r^g \in \left[\mu_{RC} - m\sigma_{RC,\infty},\ \mu_{RC} + m\sigma_{RC,\infty}\right],$$

typically with $m = 3$. If the grid is equally spaced, then

$$\Delta = \frac{2m\sigma_{RC,\infty}}{G-1}, \qquad r^g = \mu_{RC} - m\sigma_{RC,\infty} + (g-1)\Delta.$$

Given the current node r^g, the conditional mean of next period's replacement cost is

$$\mu_g = \rho_0 + \rho_1 r^g.$$

Then Tauchen defines the discrete transition matrix $Q = [Q_{gh}]_{g,h=1}^G$ by assigning probability mass to each grid cell:

$$Q_{g1} = \Phi\left(\frac{r^1 - \mu_g + \Delta/2}{\sigma_\rho}\right),$$

$$Q_{gh} = \Phi\left(\frac{r^h - \mu_g + \Delta/2}{\sigma_\rho}\right) - \Phi\left(\frac{r^h - \mu_g - \Delta/2}{\sigma_\rho}\right), \qquad h = 2, \dots, G-1,$$

$$Q_{gG} = 1 - \Phi\left(\frac{r^G - \mu_g - \Delta/2}{\sigma_\rho}\right).$$

So Q_{gh} is the probability that RC_{t+1} moves from current grid point r^g into the h-th grid cell.

Once Q is constructed, the mileage transition is the same as before:

$$x' = \begin{cases} 1, & a = 1, \\ \min(x+1, 7), & a = 0. \end{cases}$$

Hence the full transition matrices become

$$F_0 = M_0 \otimes Q, \qquad F_1 = M_1 \otimes Q.$$

At that point, part (a) proceeds exactly as before: use F_0 and F_1 inside the Bellman operator, solve the fixed point for each trial θ, compute CCPs, and maximize the state-level likelihood. So Tauchen only changes the way the exogenous RC process is discretized; it does not change the NFXP logic itself.

The code below is an implementation demo and it does not produce the same result but the key detailed can be found in the github repository

[8]:
```python
import numpy as np
from scipy.special import ndtr


def tauchen_rc_transition(
    rho0: float,
    rho1: float,
    sigma_rho: float,
    G: int,
    m: float = 3.0,
):
    """
    Tauchen discretization for
        RC_t = rho0 + rho1 * RC_{t-1} + e_t,    e_t ~ N(0, sigma_rho^2)

    Returns
    -------
    rc_grid : ndarray, shape (G,)
        Tauchen grid for replacement cost.
    Q : ndarray, shape (G, G)
        Markov transition matrix on the Tauchen grid.
    """
    if G < 2:
        raise ValueError("G must be at least 2.")
    if sigma_rho <= 0:
        raise ValueError("sigma_rho must be positive.")
    if abs(rho1) >= 1:
        raise ValueError("Tauchen assumes |rho1| < 1 for stationarity.")

    # Stationary mean and standard deviation
    mu_rc = rho0 / (1.0 - rho1)
    sigma_rc_stat = sigma_rho / np.sqrt(1.0 - rho1**2)

    # Equally spaced Tauchen grid
    rc_grid = np.linspace(
        mu_rc - m * sigma_rc_stat,
        mu_rc + m * sigma_rc_stat,
        G,
        dtype=np.float64,
    )

    delta = rc_grid[1] - rc_grid[0]
    Q = np.zeros((G, G), dtype=np.float64)

    for g in range(G):
        mean_next = rho0 + rho1 * rc_grid[g]
```

```
        # Left tail
        Q[g, 0] = ndtr((rc_grid[0] - mean_next + 0.5 * delta) / sigma_rho)

        # Interior bins
        for h in range(1, G - 1):
            upper = (rc_grid[h] - mean_next + 0.5 * delta) / sigma_rho
            lower = (rc_grid[h] - mean_next - 0.5 * delta) / sigma_rho
            Q[g, h] = ndtr(upper) - ndtr(lower)

        # Right tail
        Q[g, G - 1] = 1.0 - ndtr((rc_grid[G - 1] - mean_next - 0.5 * delta) /␣
↪sigma_rho)

    # Numerical guard
    Q /= Q.sum(axis=1, keepdims=True)

    return rc_grid, Q
```